烟幕干扰弹的投放策略

摘要

烟幕干扰弹作为现代特种弹药的重要组成部分，在战场遮蔽、目标干扰和战术掩护中发挥着关键作用。在无人机投掷烟雾干扰弹的现实场景中，通过合理设计无人机投弹的方向、速度、投弹与引爆时间，可以在烟幕弹数量一定的情况下针对来袭导弹，尽可能延长对目标物的“有效遮蔽时间”，起到更好的保护作用。本题将投掷烟雾弹，产生干扰云团隐藏目标的过程抽象成数学模型，在无人机数量，投弹数量已知的情况下，面对不同数量的导弹来袭，研究能产生最长“有效遮蔽时间”的投弹策略，为如何更好的保护目标同时节约战略资源提供了有效的帮助

针对问题一，无人机以一已知速度和方向运动，在已知时刻投下和引爆烟幕干扰弹，目的是遮蔽一已知速度和方向的来袭导弹。本文充分考虑圆柱体的几何形状和空间投影，引入导弹关于干扰云团的“视线锥”模型及其在真目标圆柱上下底面的投影，将遮蔽问题转化为判定一个圆是否完全包含在特定圆锥曲线的计算几何问题，最终得到时间为 **1.40s**；并给出另一种取圆柱上充分多离散点取样的建模，大大降低计算量，求得结果为 **1.39s**, 误差仅为 1%，为后续求解建立一个简化但不失精度的模型

针对问题二，已知用单枚烟幕弹面对单枚导弹的情形，转化为一个四维决策变量在有约束条件下的最优化问题，优化目标为有效遮蔽时长，我们经过多种优化算法的求解尝试和原因分析，最终确立了以粒子群 (PSO) 算法为骨架的求解框架，配合对无人机飞行方向的分层遍历优化，求解出的最长有效遮蔽时间为 **4.58s**。

针对问题三，在多枚烟雾弹面对单枚导弹的情形下，延续问题二的思想，以及考虑到两枚烟幕弹联合遮蔽单枚导弹的特殊情况下对总遮蔽时间的修正，使用多种优化算法对问题进行求解，最终使用遗传算法加上对精英个体的微调优化，得出了遮蔽时间最长为 **6.60s**, 并提出“零平面”和“多峰值”的模型，分析解释了常见的粒子群算法和差分优化算法失效的原因，保证了所用理论的可靠性。

针对问题四，当多枚烟幕弹从不同无人机上掷出时，使用和问题三类似的遗传算法及其优化，并充分考虑不同无人机投弹的时间约束，通过设置较高的迭代次数和较大的种群数量，尽可能探索解空间中的可行组会，最终求得最长遮蔽时间为 **9.24s**

针对问题五，在多枚烟雾弹，多架飞机，多枚导弹的复杂情况下，转化为一个非线性多参数的优化问题，通过前几问的分析，常规的智能优化算法基本都难以在可接受的时间内收敛，因此本文仍然采用上述遗传算法的优化和大范围的种群，并在变异方式等节点加以特殊化引导，得以在可接受的时间内使算法收敛。

关键字：非线性优化　遗传算法　粒子群算法　精英个体微调　计算几何

目录

一、问题重述 ····· **5**

1.1 问题背景 ····· 5

1.2 问题提出 ····· 5

二、模型假设 ····· **6**

三、符号说明 ····· **7**

四、问题分析 ····· **7**

4.1 问题一分析 ····· 7

4.2 问题二分析 ····· 7

4.2.1决策变量 ····· 8

4.2.2主要约束条件 ····· 8

4.2.3问题复杂度分析 ····· 8

4.3 问题三分析 ····· 8

4.3.1决策变量 ····· 8

4.3.2主要约束条件 ····· 9

4.3.3目标函数构建 ····· 9

4.3.4问题复杂度分析 ····· 9

4.4 问题四分析 ····· 9

4.4.1决策变量 ····· 10

4.4.2主要约束条件 ····· 10

4.4.3目标函数构建 ····· 10

4.4.4问题复杂度分析 ····· 10

4.5 问题五分析 ····· 10

4.5.1决策变量 ····· 11

4.5.2主要约束条件 ····· 11

4.5.3目标函数构建 ····· 11

4.5.4问题复杂度分析 ····· 11

五、问题建模与求解 ····· **12**

5.1 问题一 ····· 12

5.1.1基本运动与物理模型 ····· 12

5.1.2 几何遮蔽模型："视线锥" · · ·

6.1.3 问题三至五 · · ·

一、 问题重述

1.1 问题背景

烟幕干扰弹作为现代特种弹药的重要组成部分，在战场遮蔽、目标干扰和战术掩护中发挥着关键作用。其通过高效生成烟雾，干扰敌方观测和制导系统，为作战行动提供重要支持[1]。随着技术的进步，现已可以在抛撒前精确控制烟幕干扰弹到达预定位置，通过时间引信时序控制起爆时间。基于此，现考虑运用无人机完成一种有效遮蔽范围为以 10m 为半径的球形云团，持续时长 20s 的烟幕干扰弹的投放策略问题。

1.2 问题提出

设计围绕单枚导弹，使用单架无人机投放至围绕多枚导弹，使用多架无人机投放烟幕干扰弹的遮蔽策略，以实现有效遮蔽时长的最大化。其中已知导弹、无人机、烟幕弹、真假目标等的初位置：假目标为原点，水平面为 xy 轴平面，真目标下底面的圆心为 (0,200,0)。3 枚导弹 M1、M2、M3 分别位于 (20000,0,2000)、(19000,600,2100)、(18000,−600,1900)；5 架无人机的位置信息分别 FY1(17800,0,1800)、FY2(12000,1400,1400)、FY3(6000,−3000,700)、FY4(11000,2000,1800)、FY5(13000,−2000,1300)。运动方向及速度等信息如下：导弹以 300m/s 的速度朝假目标匀速直线运动，无人机可以沿任意方向进行等高直线运动，速度为 70-140m/s，干扰弹投出后在爆炸前只受重力作用，起爆后以 3m/s 匀速竖直下降，且无人机投放两枚烟幕干扰弹至少间隔 1s。建立数学模型解决如下问题 (示意图如下)：

问题一：无人机 FY1 对导弹 M1 投放一枚干扰弹实施干扰。已知 FY1 以 120m/s 的速度向假目标飞行 1.5s 后投放干扰弹，投放后 3.6s 发生爆炸，判定当导弹与真目标的任一连线与球形干扰云团相交时为有效遮蔽，根据已知条件计算干扰弹对 M1 的有效遮蔽时长。

问题二：利用 FY1 对 M1 投放一枚干扰弹实施干扰，遮蔽条件与问题一相同，要求以尽可能长的遮蔽时间为优化目标，确定 FY1 最佳的飞行方向、飞行速度、烟幕干扰弹投放点和烟幕干扰弹起爆点。

问题三：利用 FY1 投放三枚干扰弹对 M1 实施干扰，遮蔽条件与问题一相同。优化目标与第二问相同，设计投放策略，求出最佳的无人机运动方向和速度，三枚干扰弹各自的投放坐标、起爆坐标以及有效干扰时长。

问题四：利用 FY1、FY2、FY3 各投放一枚干扰弹对 M1 实施干扰，优化目标与问题二相同，求出最佳投放策略，包括三架无人机的运动速度、方向、各自投放干扰弹的坐标、干扰弹起爆坐标及有效干扰时长。

问题五：利用五架无人机对三个导弹实施干扰，每架无人机投放三枚干扰弹，优化目标与问题二相同，求出最佳投放策略，包括每架无人机的运动方向、速度、每一枚导

弹投放的坐标、起爆坐标、有效干扰时长及其干扰的导弹编号。

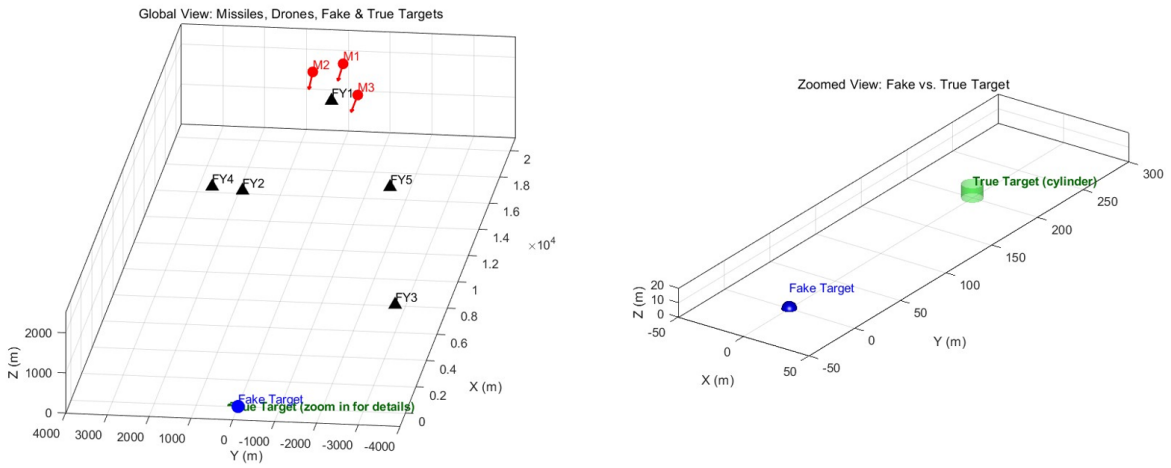

图 1　初始三维示意图

二、 模型假设

为便于问题建模与求解，对实际场景作如下合理简化与假设：

- **导弹运动假设**：忽略导弹受到的重力和空气阻力影响，认为导弹始终沿既定方向以恒定速度做匀速直线运动。
- **干扰弹运动假设**：烟幕干扰弹在投放至爆炸前仅受重力作用，视为理想平抛运动；爆炸后立即形成球形云团，其半径在有效期内保持不变，且云团球心以恒定速率垂直向下运动，忽略水平漂移。
- **无人机运动假设**：无人机在接收任务后可瞬时调整航向，随后保持恒定高度与恒定速度做直线匀速飞行。
- **烟幕相互作用假设**：不同爆炸形成的烟幕云团之间不发生化学或流体动力学上的复杂交互，其遮蔽效果在空间上直接取并集[2]。
- **遮蔽判定假设**：若导弹与真目标之间存在一条不受烟幕阻挡的直线视线，则判定为未被遮蔽；反之，则判定为有效遮蔽。
- **独立性假设**：不同导弹之间的飞行与探测互不影响；无人机之间在飞行与投放过程中不存在航路冲突与协同约束，仅需满足题目给定的最小投放间隔条件。

三、 符号说明

符号	意义
v	飞行速度（m/s）
t_0	飞行时间（s）
t_d	引爆延时（s）
t_{boom}	烟幕弹爆炸时间（s）
θ	无人机运动方向
T_{eff}	有效遮蔽时长（s）
P	物体坐标

四、 问题分析

4.1 问题一分析

第一题要求在给定的无人机投放策略下（飞行速度 $v = 120$ m/s，飞行时间 $t_0 = 1.5$ s，引爆延迟 $t_d = 3.6$ s），计算烟幕对 M1 导弹的有效遮蔽时长。核心思路是建立一个能够精确描述“完全遮蔽”状态的几何模型。简单的单点视线遮蔽模型较为粗糙，我们采用一种更精细的“体遮蔽”思想：仅当整个目标圆柱体都位于由导弹和烟幕云团形成的“视线盲区”内时，才判定为有效遮蔽。

我们将这个视线盲区在空间中建模为一个圆锥体（视线锥），其顶点为导弹，并与烟幕云团球体相切。目标的遮蔽状态取决于其所在的水平空间是否被这个视线锥完全覆盖。

根据导弹与云团的相对高度，遮蔽锥与水平面会产生不同性质的截面（椭圆或双曲线[3]），因此我们将整个过程分为两个阶段进行讨论，并容易知道当导弹高度低于云团最低点后，遮蔽必定失效，计算终止。

4.2 问题二分析

本次任务的核心目标是为无人机（FY1）规划一套最优的烟幕弹投放策略，以**最大化对地面圆柱体目标的有效遮蔽时长** (T_{eff})，从而对抗来袭的导弹（M1）。

4.2.1 决策变量

投放策略主要包括无人机的飞行方向、飞行速度以及烟幕弹的投放点和起爆点。这些可以量化为 4 个核心决策变量:

- 无人机 FY1 的飞行方向角 θ。
- 无人机 FY1 的飞行速度 v。
- 烟幕弹的投弹时间 t_0
- 烟幕弹的引爆延迟 t_d。

4.2.2 主要约束条件

- 无人机 FY1 的初始位置为 $(17800, 0, 1800)$ 。无人机等高度匀速直线飞行，速度范围为 70m/s 至 140m/s 。
- 烟幕弹在重力作用下运动，引爆后形成半径为 10m、以 3m/s 匀速下沉的球状云团，有效遮蔽时间为起爆后 20s 。
- 导弹 M1 从 $(20000, 0, 2000)$ 沿直线飞向假目标原点 $(0, 0, 0)$，速度为 300m/s，用时约 67.00s.

4.2.3 问题复杂度分析

这是一个四维、连续、非线性的多变量优化问题[4]。其目标函数（有效遮蔽时长 T_{eff}）没有简单的解析表达式，必须通过对一个动态过程进行模拟计算得出。这种特性使得问题非常复杂，传统的解析方法难以求解。因此，采用粒子群（PSO）这类不依赖梯度信息的智能优化算法[4]，从而在这种“黑箱”解空间中寻找高质量的解。

4.3 问题三分析

本次任务要求我们利用无人机 FY1 投放三枚烟幕干扰弹，以干扰来袭导弹 M1，核心目标是使烟幕对位于 $(0, 200, 0)$ 的圆柱形真目标的有效遮蔽时间尽可能长。这本质上是一个多变量、非线性的优化问题。

4.3.1 决策变量

投放策略主要包括无人机的飞行方向、飞行速度以及每枚烟幕弹的投放点和起爆点。这些可以量化为 8 个核心决策变量:

- 无人机 FY1 的飞行方向角 θ。
- 无人机 FY1 的飞行速度 v。
- 三枚烟幕弹的投弹时间 $t_{0,1}, t_{0,2}, t_{0,3}$。

- 三枚烟幕弹的引爆延迟 $t_{d,1}, t_{d,2}, t_{d,3}$。

4.3.2 主要约束条件

- 无人机 FY1 的初始位置为 $(17800, 0, 1800)$ 。无人机等高度匀速直线飞行，速度范围为 70m/s 至 140m/s 。
- 每枚烟幕弹的投放至少间隔 1 秒。这意味着 $t_{0,2} \geq t_{0,1} + 1.0$ 和 $t_{0,3} \geq t_{0,2} + 1.0$。
- 烟幕弹在重力作用下运动，引爆后形成半径为 10m、以 3m/s 匀速下沉的球状云团，有效遮蔽时间为起爆后 20s 。
- 导弹 M1 从 $(20000, 0, 2000)$ 沿直线飞向假目标原点 $(0, 0, 0)$，速度为 300m/s，用时约 67.00s.

4.3.3 目标函数构建

- 核心目标是最大化总有效遮蔽时长。这要求我们精确计算在任意时刻，$t \geq 0$，导弹 M1 的视线是否被烟幕云团完全遮蔽。
- 导弹 M1 的视线是其当前位置到圆柱形真目标上所有点的连线。由于真目标是一个圆柱，我们可以在其上下底面选取足够多的离散点来近似模拟。
- 判据：在任意时刻 t，若从导弹当前位置到真目标上所有离散点的视线都被至少一枚有效烟幕云团遮挡，则该时刻被视为有效遮蔽时刻。
- 总有效遮蔽时长即为所有有效遮蔽时刻的时间累加。

4.3.4 问题复杂度分析

与问题 2 类似，问题 3 也是一个连续、非线性的多变量优化问题，但维数从 4 维上升到了 8 维，进一步加大了问题的复杂性。在问题 2 中使用的粒子群（PSO）在进行高维变量优化时存在抽样效果差、收敛速度慢等问题，需要引入更适合解决高维的差分进化算法（DE）、遗传算法（GA）等。

4.4 问题四分析

本次任务要求我们使用 FY1,FY2,FY3 三架无人机各投一枚烟幕弹对 M1 进行干扰，核心目标仍是使烟雾对位于真目标点 (0,200,0) 的圆柱体有效遮蔽时间尽可能长。这本质上是三个问题二模型的复合，但需要注意多枚烟雾弹之间的关系，即多枚烟雾弹可以同时对目标进行遮蔽。

4.4.1 决策变量

投放策略主要包括无人机的飞行方向、飞行速度以及每枚烟幕弹的投放点和起爆点。这些可以量化为 12 个核心决策变量：

- 无人机 FY1,FY2,FY3 的飞行方向角 $\theta_1, \theta_2, \theta_3$。
- 无人机 FY1,FY2,FY3 的飞行速度 v_1, v_2, v_3。
- 三枚烟幕弹的投弹时间 $t_{0,1}, t_{0,2}, t_{0,3}$。
- 三枚烟幕弹的引爆延迟 $t_{d,1}, t_{d,2}, t_{d,3}$。

4.4.2 主要约束条件

- 无人机 FY1,FY2,FY3 的初始位置分别为 $(17800, 0, 1800), (12000, 1400, 1400), (6000, -3000, 700)$ 。无人机等高度匀速直线飞行，速度范围为 70m/s 至 140m/s 。
- 烟幕弹在重力作用下运动，引爆后形成半径为 10m、以 3m/s 匀速下沉的球状云团，有效遮蔽时间为起爆后 20s 。
- 导弹 M1 从 $(20000, 0, 2000)$ 沿直线飞向假目标原点 $(0, 0, 0)$，速度为 300m/s，用时约 67.00s.

4.4.3 目标函数构建

- 核心目标是最大化总有效遮蔽时长。这要求我们精确计算在任意时刻，$t \geq 0$，导弹 M1 的视线是否被所有的烟幕云团完全遮蔽（可以由多颗烟雾云团共同遮蔽）。
- 导弹 M1 的视线是其当前位置到圆柱形真目标上所有点的连线。由于真目标是一个圆柱，我们可以在其上下底面选取足够多的离散点来近似模拟。
- 判据：在任意时刻 t，若从导弹当前位置到真目标上所有离散点的视线都被至少一枚有效烟幕云团遮挡，则该时刻被视为有效遮蔽时刻。
- 总有效遮蔽时长即为所有有效遮蔽时刻的时间累加。

4.4.4 问题复杂度分析

问题 4 依然是一个连续、非线性的多变量优化问题，维数为 12，复杂性与问题 3 相当，我们考虑使用和问题 3 类似的算法解决问题 4，即遗传算法等适合解决高维、连续、非线性多变量问题的算法[5]。

4.5 问题五分析

最后的任务要求我们使用 FY1,FY2,FY3,FY4,FY5 五架无人机各投三枚烟幕弹对 M1,M2,M3 三枚导弹进行干扰，核心目标是使烟雾对位于真目标点 (0,200,0) 的圆柱体有

效遮蔽时间尽可能长。这本质上是多个问题 2 模型的复合，由于引入了多个无人机、多颗烟雾弹和多个导弹，问题的复杂性相较前几题有了进一步显著提升。

4.5.1 决策变量

投放策略主要包括无人机的飞行方向、飞行速度以及每枚烟幕弹的投放点和起爆点。这些可以量化为 40 个核心决策变量：

- 无人机 FY1,FY2,FY3,FY4,FY5 的飞行方向角 $\theta_1, \theta_2, \theta_3, \theta_4, \theta_5$。
- 无人机 FY1,FY2,FY3,FY4,FY5 的飞行速度 v_1, v_2, v_3, v_4, v_5。
- 每架无人机所携带的三枚烟幕弹的投弹时间 $t_{0,1,1}, t_{0,1,2}, t_{0,1,3}, t_{0,2,1}, t_{0,2,2}, t_{0,2,3}, \ldots, t_{0,5,3}$。
- 每架无人机所携带的三枚烟幕弹的引爆延迟 $t_{d,1,1}, t_{d,1,2}, t_{d,1,3}, t_{d,2,1}, t_{d,2,2}, t_{d,2,3}, \ldots, t_{d,5,3}$。

4.5.2 主要约束条件

- 无人机 FY1,FY2,FY3,FY4,FY5 的初始位置分别为 $(17800, 0, 1800), (12000, 1400, 1400), (6000, -3000, 700), (11000, 2000, 1800), (13000, -2000, 1300)$。无人机等高度匀速直线飞行，速度范围为 70m/s 至 140m/s 。
- 每枚烟幕弹的投放至少间隔 1 秒。即 $t_{0,i,2} \geq t_{0,i,1} + 1.0, t_{0,i,3} \geq t_{0,i,2} + 1.0, i \in \{1, 2, 3, 4, 5\}$。
- 烟幕弹在重力作用下运动，引爆后形成半径为 10m、以 3m/s 匀速下沉的球状云团，有效遮蔽时间为起爆后 20s 。
- 导弹 M1,M2,M3 分别从 $(20000, 0, 2000), (19000, 6000, 2100), (18000, -600, 1900)$ 沿直线飞向假目标原点 $(0, 0, 0)$，速度为 300m/s，用时分别约为 67.00s,63.75s,60.37s.

4.5.3 目标函数构建

- 核心目标是最大化总有效遮蔽时长。这要求我们精确计算在任意时刻，$t \geq 0$，导弹的视线是否被烟幕云团完全遮蔽。
- 导弹的视线是其当前位置到圆柱形真目标上所有点的连线。由于真目标是一个圆柱，我们可以在其上下底面选取足够多的离散点来近似模拟。
- 判据：在任意时刻 t，若从导弹当前位置到真目标上所有离散点的视线都被至少一枚有效烟幕云团遮挡，则该时刻被视为有效遮蔽时刻。
- 总有效遮蔽时长即为所有导弹有效遮蔽时刻的累加。

4.5.4 问题复杂度分析

问题 5 是一个 40 维的连续、非线性的多变量优化问题，复杂性远高于前面的问题，我们考虑使用和问题 3,4 类似的算法解决问题 5，但需要在算法之上进行一定的改进，

如调整取样策略、在训练过程中进行微调、调整算法的各类参数等，同时对程序进行优化，通过批量计算、矩阵计算等方案加速算法求解。

五、 问题建模与求解

5.1 问题一

5.1.1 基本运动与物理模型

- **真目标圆柱体**: 底面圆心位于 $(0, 200, 0)$，半径 $R_{\text{target}} = 7\,\text{m}$，高 $H_{\text{target}} = 10\,\text{m}$。其所占据的底面圆区域满足方程：

$$x^2 + (y - 200)^2 \leq 7^2 \tag{1}$$

- **烟幕云团**: 被建模为一个半径 $R_{\text{cloud}} = 10\,\text{m}$ 的球体，其有效性从引爆时刻 t_{boom} 开始，持续 20 秒。
- **运动轨迹**: 导弹 $\vec{P}_{\text{M1}}(t)$ 和云团中心 $\vec{P}_{\text{cloud}}(t)$ 的运动轨迹遵循先前模型所建立的动力学方程。

5.1.2 几何遮蔽模型："视线锥"

在任意时刻 t，我们将遮蔽区域定义为一个以导弹瞬时位置为顶点的三维圆锥，该圆锥的表面与烟幕云团球体表面相切。

- **顶点**: 导弹位置 $\vec{P}_{\text{M1}}(t)$。
- **轴线**: 连接导弹中心 $\vec{P}_{\text{M1}}(t)$ 与云团中心 $\vec{P}_{\text{cloud}}(t)$ 的直线。
- **开口半角** $\alpha(t)$: 由导弹与云团的相对位置动态决定，满足：

$$\sin(\theta(t)) = \frac{R_{\text{cloud}}}{|\vec{P}_{\text{M1}}(t) - \vec{P}_{\text{cloud}}(t)|} \tag{2}$$

5.1.3 有效遮蔽的判定准则

有效遮蔽的充要条件是：视线锥与目标圆柱体所在的两个关键水平面（上底面 $z = 10$ 和下底面 $z = 0$）所产生的截面，必须同时完全包含目标圆柱体的底面圆。

我们根据导弹的 z 坐标 $P_{\text{M1},z}(t)$ 和云团的 z 坐标范围 $[P_{\text{cloud},z}(t) - R_{\text{cloud}}, P_{\text{cloud},z}(t) + R_{\text{cloud}}]$ 的相对关系，分两阶段建模。

第一阶段：导弹高于云团上顶点

当导弹的垂直位置高于整个云团球体时，即：

$$P_{\text{M1},z}(t) > P_{\text{cloud},z}(t) + R_{\text{cloud}} \tag{3}$$

此时，水平切割平面 $z=10$ 和 $z=0$ 与视线锥的截面均为椭圆[6]，我们分别记为 O_1 和 O_2。以下来推导这两个椭圆方程：

假设某时刻导弹坐标 $P_0(x_0,y_0,z_0)$，云团圆心坐标 $P_c(x_1,y_1,z_1)$，根据下图2，设圆锥上任一点 $P_m(a,b,c)$，圆锥面方程为：

$$\vec{P_0P_c}\cdot\vec{P_0P_m}=(x_0-a,y_0-b,z_0-c)\cdot(x_0-x_1,y_0-y_1,z_0-z_1)=|\vec{P_0P_c}|\cdot|\vec{P_0P_m}|\cdot\cos\theta \tag{4}$$

分别用平面 $z=0$ 和 $z=10$ 截这个圆锥面，得到椭圆上任一点 (x,y) 满足：

$$(x_0-x,y_0-y,z_0-z)\cdot(x_0-x_1,y_0-y_1,z_0-z_1)$$

$$=\sqrt{(x_0-x)^2+(y_0-y)^2+(z_0-z)^2}\cdot\sqrt{(x_0-x_1)^2+(y_0-y_1)^2+(z_0-z_1)^2}\cdot\cos\theta$$

$$=|\vec{P_0}-\vec{P_c}|\cdot|\vec{P_0}-\vec{P_m}|\cdot\cos\theta$$

然后我们使用序列最小二乘规划 (SLSQP) 算法求解椭圆到上下底面圆心的距离函数 $d(x,y)=\sqrt{x^2+(y-200)^2}$ 的最小值并与半径 7m 进行比较

- **判定条件 (如下图2所示)**: 椭圆 O_1 必须完全包含目标顶面圆，**并且**椭圆 O_2 也必须完全包含目标底面圆，即 $min\{d(x,y)\}\geq 7$

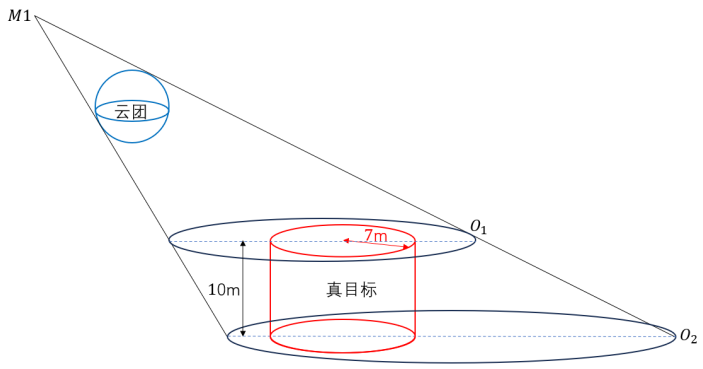

图 2　圆台包含圆柱

第二阶段：导弹高度与云团交错

当导弹的垂直位置进入云团球体的 z 坐标区间内时，即：

$$P_{\text{cloud},z}(t)-R_{\text{cloud}}\leq P_{\text{M1},z}(t)\leq P_{\text{cloud},z}(t)+R_{\text{cloud}} \tag{5}$$

此时，视线锥的顶点与部分锥身位于切割平面之间，截面不再是封闭的椭圆，而是双曲线一支形成的开放区域[3]。

- **判定条件**: 目标底面圆必须完全被这个双曲线区域所包含。

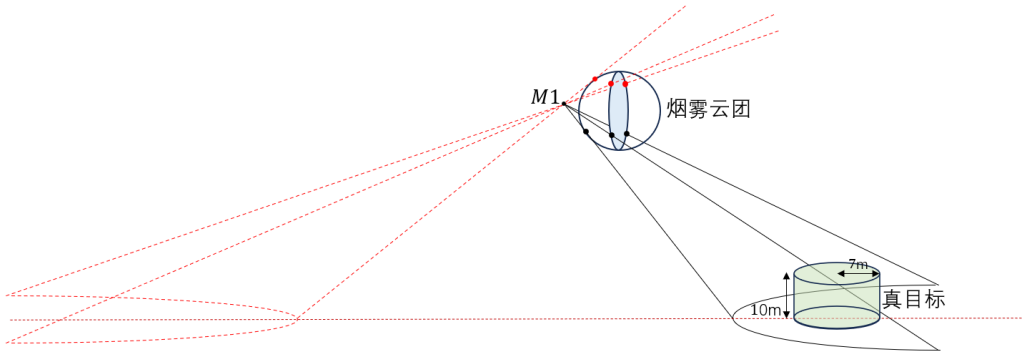

图 3　单边双曲线包含圆柱

5.1.4 包含关系判定

无论是“椭圆包含圆”还是“开放区域包含圆”，其判定都可以通过判断圆心是否在区域内，并且圆心到椭圆边界的最短距离是否大于等于圆半径来近似求解。

5.1.5 求解流程

为了计算满足上述条件的总时长，我们采用时间离散化的模拟方法。

1. **初始化**: 设定无人机飞行策略参数 (v, t_0, t_d)，计算出引爆时刻 t_{boom} 和烟幕失效时刻 $t_{\text{exp}} = t_{\text{boom}} + 20\text{s}$。初始化有效时长 $T_{\text{eff}} = 0$。
2. **时间离散化**: 设定一个微小的时间步长 dt，创建一个从 t_{det} 到 t_{exp} 的时间序列。
3. **循环计算**: 对时间序列中的每一个时刻 t 执行以下步骤:
   (a) 计算该时刻的导弹位置 $\vec{P}_{\text{M1}}(t)$ 和云团中心位置 $\vec{P}_{\text{cloud}}(t)$。
   (b) 根据两者的 z 坐标，判断当前属于第一阶段还是第二阶段。
   (c) **若为第一阶段**：计算出椭圆 O_1 和 O_2 的方程。分别进行两次“椭圆包含圆”的几何判断，即比较圆心到椭圆上任一点的最短距离 $min_{(x,y)\in\textbf{椭圆}O_i}\sqrt{x^2+(y-200)^2}$ 与 10m 的大小关系，若小于 10m 则无法完全遮盖，否则可以完全遮盖。若两次判断均成立，则 $T_{\text{eff}} = T_{\text{eff}} + dt$。
   (d) **若为第二阶段**：计算出双曲线区域的方程，同理进行圆心到双曲线最短距离 $min_{(x,y)\in\textbf{双曲线}}\sqrt{x^2+(y-200)^2}$ 与 10m 的大小比较，若判断成立，则 $T_{\text{eff}} = T_{\text{eff}} + dt$。
   (e) 判断导弹 z 坐标是否低于云团最低点，若是，则提前终止循环。
4. **输出结果**: 我们取迭代步长为 0.1 时，得到的 T_{eff} 即在该投放策略下的总有效遮蔽时长为 **1.4s**。
   整个过程中:
   烟雾弹抛下位置:[17620.0000，0，1800.0000]；

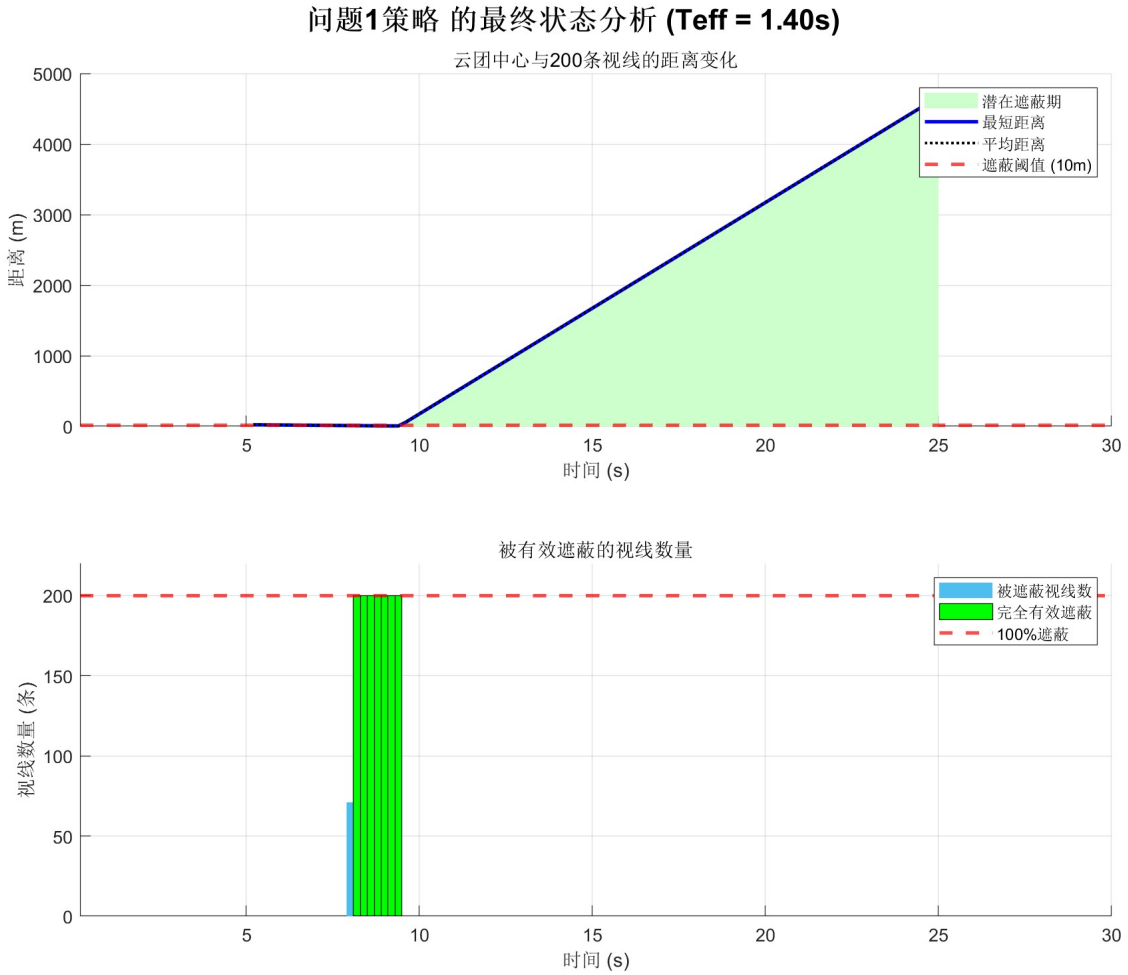

图 4　投弹全流程可视化图表

烟雾弹爆炸位置:[17188.0000，0，1736.4960]

导弹方向:[-0.99503719，0，-0.09950372]

烟雾弹爆炸时导弹位置:[18477.5931，0，1847.7593]

5.1.6 离散化计算方法

在建模过程中我们发现，采用对圆柱上下底面圆周上取若干离散点的计算方法，即将这些点与导弹用线段连接，形成一线段簇，他们必须全部穿过烟雾云团，才视为“有效遮蔽”，这一方法相对于上述的精确建模，计算量大大降低，我们在取上下底面圆周各 100 个点的情况下进行了计算，得到有效遮蔽时长为 **1.390s**, 与上述建模所得结果相比，误差为 1%，在可接受范围内，因此，后续题目中我们采用此种方法判定有效遮蔽，以大幅缩减计算量。

5.2 问题二

无人机，导弹，目标等物体的位置及运动示意图如下：

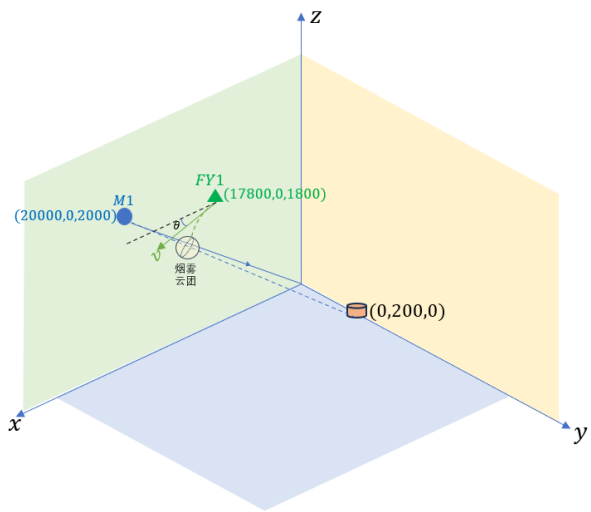

图 5　问题二示意图

5.2.1 运动学模型

1. 导弹 M1 的运动轨迹导弹 M1 从初始位置 $\boldsymbol{P}_{M1,0} = (20000, 0, 2000)$ 以 300m/s 的速度飞向原点 $\boldsymbol{P}_{Target} = (0, 0, 0)$ 。其单位方向向量为：

$$\boldsymbol{u}_{M1} = \frac{\boldsymbol{P}_{Target} - \boldsymbol{P}_{M1,0}}{||\boldsymbol{P}_{Target} - \boldsymbol{P}_{M1,0}||}$$

导弹 M1 在时刻 t 的位置 $\boldsymbol{P}_{M1}(t)$ 为：

$$\boldsymbol{P}_{M1}(t) = \boldsymbol{P}_{M1,0} + \boldsymbol{u}_{M1} \cdot V_{M1} \cdot t$$

其中 $V_{M1} = 300\text{m/s}$。导弹到达时间 $T_{M1,arrival} = ||\boldsymbol{P}_{M1,0}||/V_{M1}$。

2. 无人机 FY1 的运动轨迹无人机 FY1 从初始位置 $\boldsymbol{P}_{FY1,0} = (17800, 0, 1800)$ 等高度匀速直线飞行。设其水平飞行方向角为 θ，速度为 v。其速度向量为 $\boldsymbol{V}_{FY1} = (v\cos\theta, v\sin\theta, 0)$。无人机在时刻 t 的位置 $\boldsymbol{P}_{FY1}(t)$ 为：

$$\boldsymbol{P}_{FY1}(t) = \boldsymbol{P}_{FY1,0} + \boldsymbol{V}_{FY1} \cdot t$$

3. 烟幕弹的运动轨迹烟幕弹在时刻 t_{drop} 从无人机上投放，其初始速度与无人机速度相同。其轨迹为平抛运动，受重力影响。烟幕弹在时刻 t 的位置 $\boldsymbol{P}_{bomb}(t)$ 为：

$$\boldsymbol{P}_{cloud}(t) = \boldsymbol{P}_{FY1}(t_0) + \boldsymbol{V}_{FY1}(t_0) \cdot (t - t_0) + \frac{1}{2}\boldsymbol{g}(t - t_0)^2$$

其中 $\boldsymbol{g} = (0, 0, -9.8)$.

烟幕弹在引爆延迟 t_d 后，即时刻 $t_{boom} = t_0 + t_d$ 引爆。引爆点位置 $\boldsymbol{P}_{boom}$ 为 $\boldsymbol{P}_{cloud}(t_{boom})$。云团中心在时刻 $t \geq t_{boom}$ 的位置 $\boldsymbol{P}_{cloud}(t)$ 为：

$$\boldsymbol{P}_{cloud}(t) = \boldsymbol{P}_{boom} - (0, 0, V_{fall} \cdot (t - t_{boom}))$$

其中 $V_{fall} = 3\text{m/s}$.

5.2.2 有效遮蔽判据

我们采用圆柱上下底面的离散点来代表真目标。在每个时刻 t，如果从导弹 M1 位置 $\boldsymbol{P}_{M1}(t)$ 到真目标上任意一点 $\boldsymbol{P}_{target}$ 的视线都与此时刻烟雾云团相交，则该时刻是有效遮蔽时刻。

简化判据：若烟幕云团中心 $\boldsymbol{P}_{cloud}(t)$ 到线段 $\overline{\boldsymbol{P}_{M1}(t)\boldsymbol{P}_{target}}$ 的最短距离小于等于烟幕云团半径 $R_{cloud} = 10\text{m}$，则该点被遮蔽。

5.2.3 优化模型

目标函数: 最大化总有效遮蔽时间 T_{eff}。

$$\max \quad T_{eff} = \int_{t_{0,1}}^{max\{t_{boom,i}\}+20} I(\text{真目标完全被遮蔽})\, dt$$

其中 I 为指示函数，$max\{t_{boom,i}\} + 20s$ 为烟雾弹失效的时间。

决策变量:

$$\boldsymbol{x} = (\theta, v, t_0, t_d)$$

约束条件:

1. 无人机速度范围：$70 \leq v \leq 140$.
2. 投弹时间范围：$t_0 \geq 0$.
3. 引爆延迟范围：$t_d \geq 0$.
4. 烟幕弹飞行时间范围：$t_{fly} = t_0 + t_d$，且 $\min t_{fly} \leq t_{\text{M1_arrival}} = 67.00s$

5.2.4 分层优化策略

由于解空间维度复杂且目标函数为“黑箱”，我们采用分层优化策略来求解。

- **外层循环**：将无人机的飞行方向 θ 在可行域 $[0, \pi]$ 内进行离散化，分解为多个独立的子问题。
- **内层优化**：对于每个固定的 θ 值，问题被降维为求解 (v, t_0, t_d) 三个参数的最优组合。

这种分层优化方法具有明显优势：首先，将四维参数空间（θ, v, t_0, t_d）拆分为“外层单变量离散化 + 内层三维连续优化”，有效缩小了单次优化的搜索维度，降低了粒子群在高维空间陷入局部最优的风险[7]；其次，外层基于 θ 的规律性离散划分，使得每个子问题的优化目标更加集中（在固定方向下仅需调整投放参数），从而提升了优化结果的针对性与解释性。

5.2.5 粒子群优化算法 (PSO) 实现

我们使用粒子群算法（PSO）完成内层优化[8]，流程如下图所示：

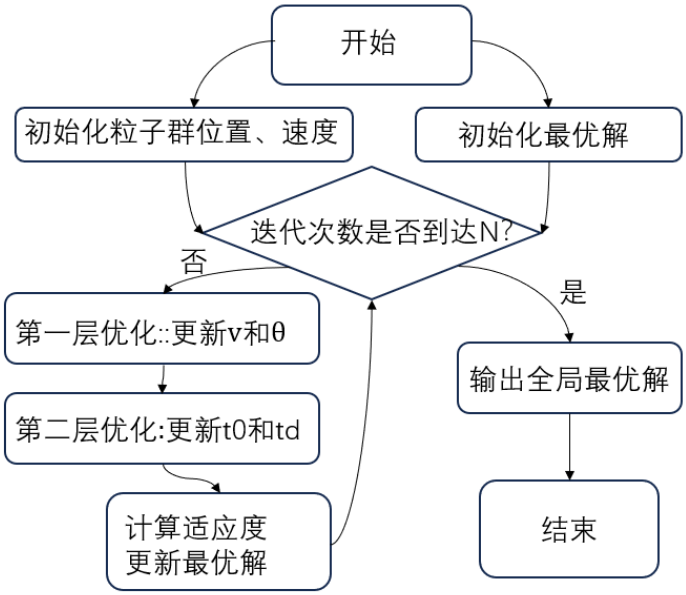

图 6　PSO 算法流程图

1. **初始化**：在三维参数空间内，随机生成一群“粒子”，每个粒子代表一套投放策略 (v, t_0, t_d)。
2. **评估**：对每个粒子，调用问题建模部分建立的模型，计算其对应的有效遮蔽时长 T_{eff}，作为其“适应度”（fitness）[9]。
3. **更新**：在每次迭代中，每个粒子会根据它自己历史上的最优位置（个体经验）和整个种群历史上的最优位置（群体智慧）来调整自己的“飞行”速度和方向，从而在解空间中进行搜索[9]。
4. **收敛**：经过足够多的迭代次数，整个粒子群会逐渐聚集在能产生最长 T_{eff} 的区域附近，从而找到当前 θ 值下的最优解。下图集展示该算法得出最优结果的收敛过程。

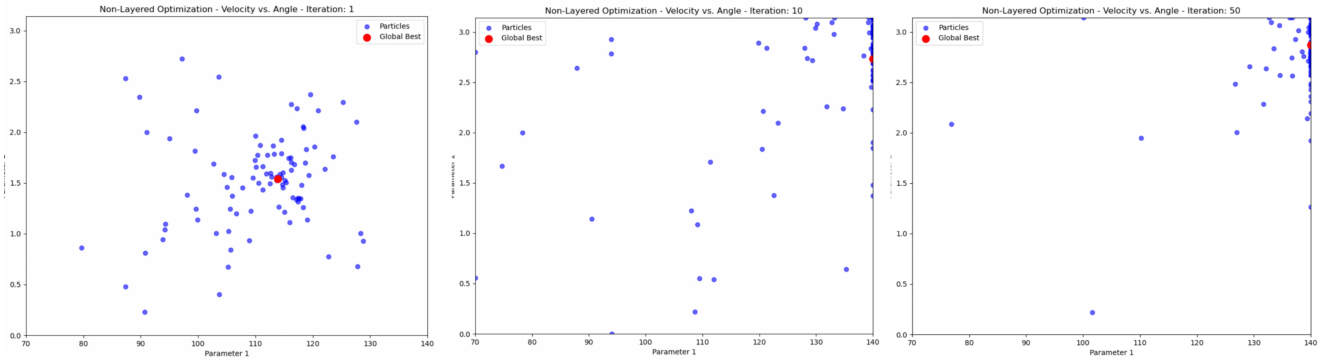

图 7　收敛效果

5.2.6 全局最优解

最后，我们比较所有外层循环（即所有 θ 值）中找到的局部最优解，其中 T_{eff} 最大的一个，即为我们最终求得的全局最优投放策略对应的有效遮蔽时间为 **4.575s**，算法收敛过程和具体遮蔽情况如下图[10]所示：

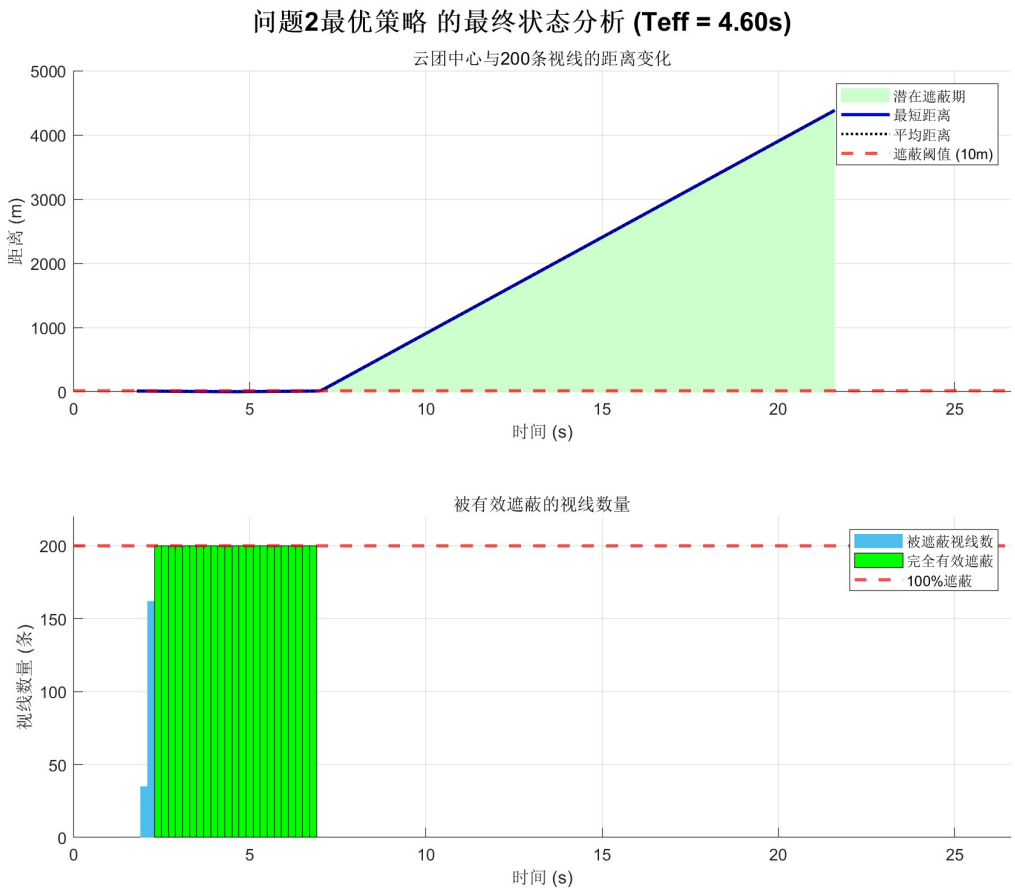

图 8　投弹最优策略可视化图表

5.2.7 模型验证与补充

为了验证我们算法的合理性，我们采用分层式优化 + 蒙特卡洛算法[11]对可行空间中不同变量组的取值进行了 20000 次随机采样，得到的最长遮蔽时间为 4.565s，与我们上面的答案相近，侧面验证了我们答案的准确性，具体信息如表1。

表 1　PSO 与蒙特卡洛算法结果对比

对比维度	蒙特卡洛算法	粒子群优化算法
速度 (m/s)	101.15	70
方向 (°)	6	3.887
投弹时间 (s)	0.39	1.776551
爆炸延时 (s)	0.39	0
遮蔽时间 (s)	**4.565**	**4.575**

由两种算法的结果对比可知，在步长同为 0.005 的情况下，使用 PSO 算法得到的方案其遮蔽时长相较于蒙特卡洛算法，**进步了 0.01 秒**，能够体现出 PSO 算法的优势；其次，就算法本身而言，PSO 算法可以利用粒子的个体最优位置和全局最优位置信息，迅速调整搜索方向，减少不必要的搜索空间，**计算复杂度降低**，而蒙特卡洛算法需要进行大量的试验才能逐渐逼近最优解。

5.3 问题三

5.3.1 运动学模型

导弹 M1 的运动轨迹，无人机 FY1 的运动轨迹，烟幕弹的运动轨迹均与问题二相同。

5.3.2 有效遮蔽判据

我们采用圆柱上下底面的离散点来代表真目标。在每个时刻 t，如果从导弹 M1 位置 $\boldsymbol{P}_{M1}(t)$ 到真目标上任意一点 $\boldsymbol{P}_{target}$ 的视线都与此时刻区域内所有云团的并集相交，则该时刻是有效遮蔽时刻。

简化判据：若存在烟幕云团中心 $\boldsymbol{P}_{cloud}(t)$ 到线段 $\overline{\boldsymbol{P}_{M1}(t)\boldsymbol{P}_{target}}$ 的最短距离小于等于该烟幕云团半径 $R_{cloud} = 10\text{m}$，则该点被遮蔽。

若圆柱上下底面的所有离散点均被一个烟幕遮蔽，则将该时刻计入该烟幕的单独有效遮蔽时刻。

5.3.3 优化模型

目标函数: 最大化总有效遮蔽时间 T_{eff}。

$$\max \quad T_{eff} = \int_{t_{0,1}}^{max\{t_{boom,i}\}+20} I(\text{真目标完全被遮蔽})\, dt$$

其中 I 为指示函数，$max\{t_{boom,i}\} + 20s$ 为最后一枚烟雾弹失效的时间。

决策变量:

$$\boldsymbol{x} = (\theta, v, t_{0,1}, t_{d,1}, t_{0,2}, t_{d,2}, t_{0,3}, t_{d,3})$$

约束条件:

1. 无人机速度范围：$70 \leq v \leq 140$.
2. 投弹时间间隔：$t_{0,2} \geq t_{0,1} + 1.0$ 且 $t_{0,3} \geq t_{0,2} + 1.0$.
3. 投弹时间范围：$t_{0,i} \geq 0$.
4. 引爆延迟范围：$t_{d,i} \geq 0$.
5. 烟幕弹飞行时间范围：$t_{fly,i} = t_{0,i} + t_{d,i}$，且 $\min t_{fly,i} \leq t_{\text{M1_arrival}} = 67.00s$

5.3.4 模型求解与计算

由于该优化问题为多维非线性问题，且目标函数计算复杂度高，传统的梯度下降法不适用。因此，我们选择使用混合策略的遗传算法[12]进行求解。该算法通过模拟自然选择和遗传机制，能在复杂解空间中进行全局搜索，并结合局部搜索以加速收敛。

5.3.5 算法流程

1. **初始化种群**: 随机生成包含多个个体的初始种群，每个个体代表一个由 8 个决策变量构成的投放策略。确保初始个体满足基本约束条件。
2. **适应度计算**: 对种群中的每个个体,调用物理函数"$calculate_effective_time_multi_bomb$"来计算其对应的有效遮蔽时长，此时长即为该个体的适应度值。
3. **精英选择**[13]: 选出当前种群中适应度最高的若干个体（精英），给予其微小的随机扰动以探测精英个体周围是否存在更优解，并放入下一代，以保证最优解不被丢失。
4. **交叉与变异**:
   - **选择**: 使用轮盘赌选择等方法，根据适应度值从当前种群中选择父代个体。
   - **交叉**: 对选出的父代进行均匀交叉操作，生成新的子代个体。
   - **变异**: 对子代个体进行高斯变异，在参数空间内进行小幅随机扰动，以增加种群多样性。
5. **局部搜索增强**: 对每代的精英个体，进行小范围的局部搜索，通过微调其参数来进一步提升适应度值，以加速算法收敛。

6. **新种群生成**: 将精英个体和新生成的子代个体组合，形成新的种群，进入下一代迭代。
7. **终止条件**: 重复上述步骤，直到达到预设的最大迭代次数。

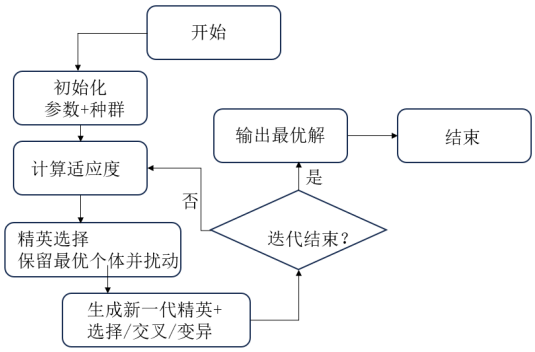

图 9　精英选择-遗传算法流程图

5.3.6 结果对比与补充说明

此外，我们采用包括**遗传算法**、**CMA 算法**、**PSO 算法**、**差分优化算法**等多种优化算法对该问进行了求解

结果如下：PSO 算法和差分优化算法难以在可接受的时间内收敛到非零解；遗传算法与 CMA 算法在相同步长 ($dt = 0.01$) 下得到的结果如下表**??**对比：

表 2　GA 与 CMA 算法的最优策略对比

参数	遗传算法 (Genetic Algorithm)	CMA 算法 (CMA Algorithm)
飞行速度 v_f (m/s)	117.29	70.0204
飞行方向 θ (度)	6.63	5.71
第一枚投弹时间 $t_{0,1}$ (s)	0	1.2270
第一个爆炸延时 $t_{d,1}$ (s)	0	0.1215
第二枚投弹时间 $t_{0,2}$ (s)	1	3.2244
第二个爆炸延时 $t_{d,2}$ (s)	0	0.9209
第三枚投弹时间 $t_{0,3}$ (s)	2	1.4479
第三个爆炸延时 $t_{d,3}$ (s)	0	1.6895
最大覆盖时间 (s)	**6.41**	**4.51**

对出现的这两种现象，我们进行了原因分析：

1. PSO 算法和差分优化算法的不收敛：问题的八维解空间存在大量局部最优解。单一参数的微小变动，可能导致烟幕云团的位置与导弹视线发生错位，使覆盖时间从一个较优值瞬间变为零。这种多峰的情况使得算法很难获得稳定的梯度信息来持续优化，极易被困在某个次优的参数组合上。

   在 8 维的参数空间中，能产生有效覆盖（即覆盖时间 > 0）的区域非常狭小且分散。算法在搜索初期，绝大多数生成的策略其覆盖时间都为零。这构成了一个巨大的“零平台”，算法无法判断更有希望的探索方向，探索过程接近于随机猜试，效率极低。具体可视化图表[10]如下，展示了算法初期产生的较大“零平台”，以及分散且繁多的局部最优点

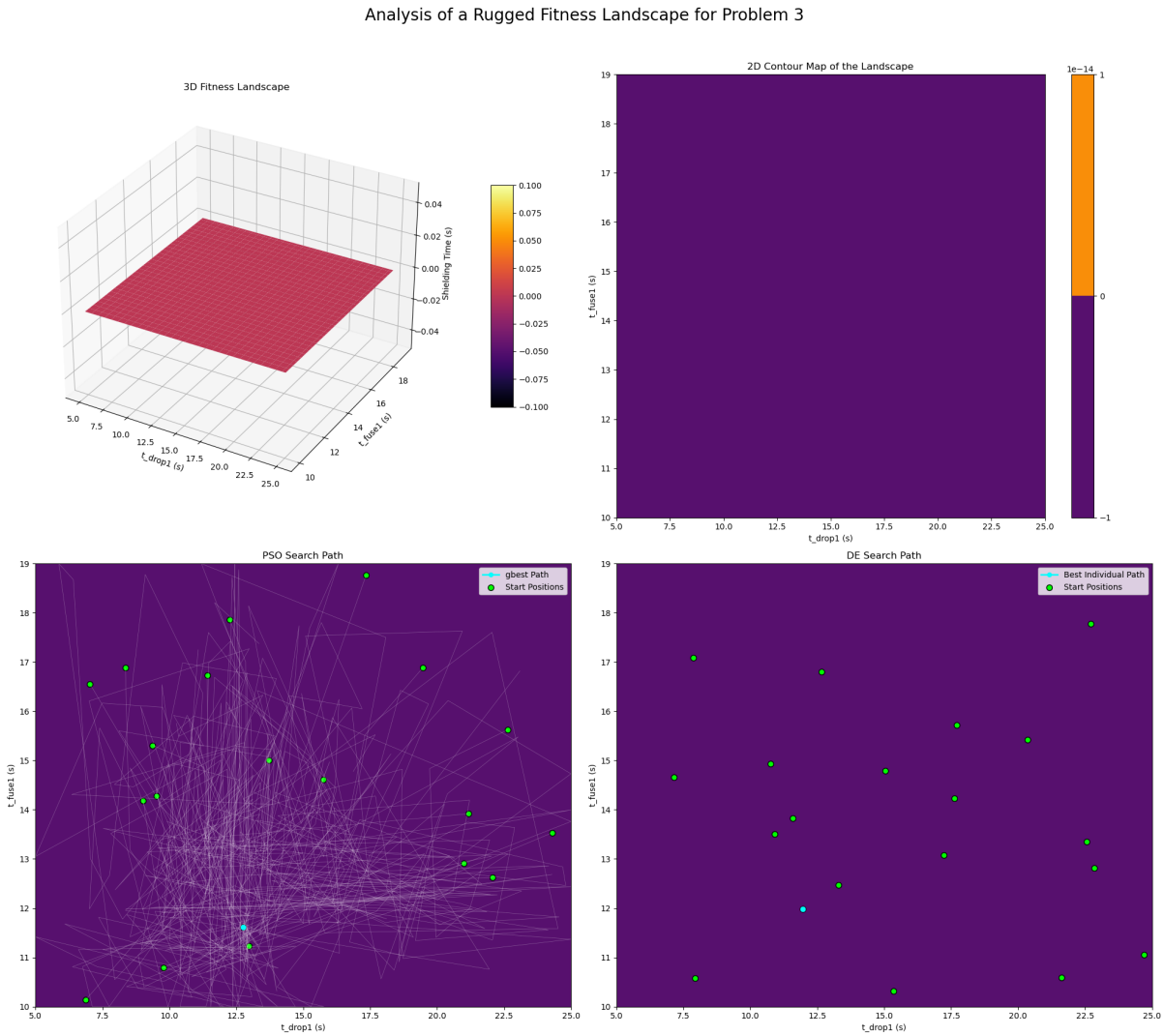

图 10　初期“零平台”及局部最优点分布

- **最大有效遮蔽时长: 6.41s**
- **无人机 FY1 飞行参数:**

- 飞行方向: 6.63 度
- 飞行速度: 117.29 m/s

- **三枚烟幕弹投放参数**:
  - 第 1 枚烟幕弹：投弹时间 0 s，引爆延迟 0 s。
  - 第 2 枚烟幕弹：投弹时间 1 s，引爆延迟 0 s。
  - 第 3 枚烟幕弹：投弹时间 2 s，引爆延迟 0 s。

最终结果已按要求保存至 **'result1.xlsx' 文件**中。

整体投弹策略可视化[10]如下：

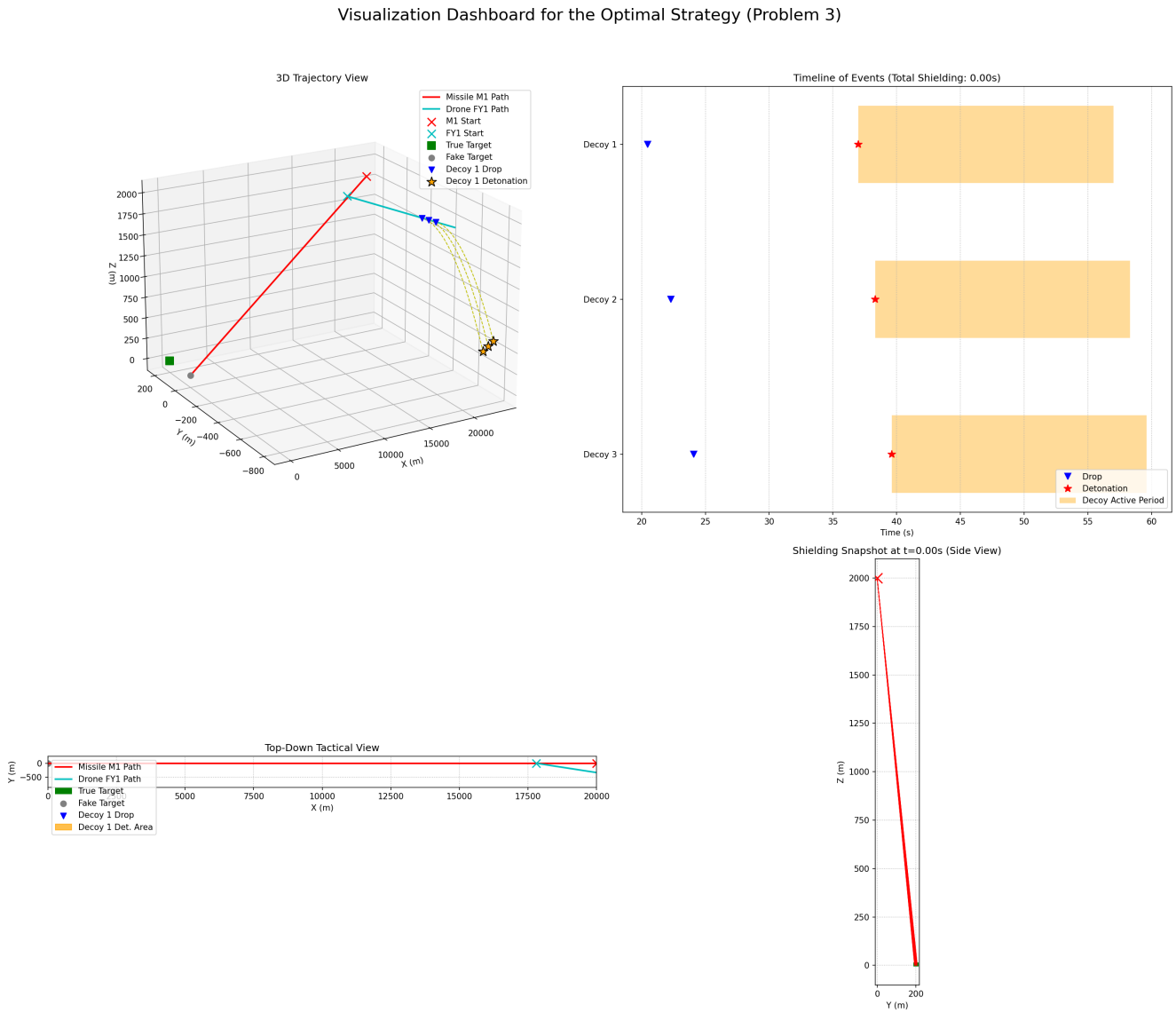

图 11　投弹策略可视化组图

3D Trajectory View (3D 空间轨迹图)

此图宏观展示了导弹、无人机与烟幕弹在三维空间中的完整运动轨迹。展示无人机将三枚烟幕弹运送至预定空间引爆点的过程。

Timeline of Events (时序甘特图)

该图表以时间为轴，精确展示了投放、引爆等关键事件的顺序。图中标示的绿色时间段为实际有效遮蔽窗口，所有这些不连续窗口的总时长构成了最终 6.6 秒的成绩。

Top-Down Tactical View (水平布局图)

这张俯视图更清晰地展现了无人机的飞行角度与烟幕屏障的水平布局。它揭示了无人机通过特定的倾斜航线，在导弹飞行路径侧前方形成有效遮蔽“墙”的战术意图。

Shielding Snapshot (遮蔽模拟)

此图为成功遮蔽瞬间的模拟。图中所有从导弹发出的视线均穿过了烟幕云（呈亮绿色），直观地证明了在该时刻，烟幕云成功阻断了导弹的全部关键视线。

5.4 问题四

5.4.1 运动学模型

导弹 M1 的运动轨迹、烟幕弹的运动轨迹均与问题二相同。

无人机 FY1,FY2,FY3 的运动轨迹无人机 FYi($i \in [1,2,3]$) 从初始位置 $\boldsymbol{P}_{FYi,0}$ 等高度匀速直线飞行。设其水平飞行方向角为 θ_i，速度为 v_i。其速度向量为 $\boldsymbol{V}_{FYi} = (v_i \cos\theta_i, v_i \sin\theta_i, 0)$。无人机在时刻 t 的位置 $\boldsymbol{P}_{FYi}(t)$ 为：

$$\boldsymbol{P}_{FYi}(t) = \boldsymbol{P}_{FYi,0} + \boldsymbol{V}_{FYi} \cdot t$$

5.4.2 有效遮蔽判据

我们采用圆柱上下底面的离散点来代表真目标。在每个时刻 t，如果从导弹 M1 位置 $\boldsymbol{P}_{M1}(t)$ 到真目标上任意一点 $\boldsymbol{P}_{target}$ 的视线都与此时刻区域内所有云团的并集相交，则该时刻是有效遮蔽时刻。

简化判据：若存在烟幕云团中心 $\boldsymbol{P}_{cloud}(t)$ 到线段 $\overline{\boldsymbol{P}_{M1}(t)\boldsymbol{P}_{target}}$ 的最短距离小于等于烟幕云团半径 $R_{cloud} = 10\text{m}$，则该点被遮蔽。

若圆柱上下底面的所有离散点均被一个烟幕遮蔽，则将该时刻计入该烟幕的单独有效遮蔽时刻。

5.4.3 优化模型

目标函数: 最大化总有效遮蔽时间 T_{eff}。

$$\max\{T_{eff}\} = \int_{t_{0,1}}^{max\{t_{boom,i}\}+20} I(\text{真目标完全被遮蔽})\, dt$$

其中 I 为指示函数，$max\{t_{boom,i}\}+20s$ 为最后一枚烟雾弹失效的时间。

决策变量:

$$\boldsymbol{x}=(\theta_1,\theta_2,\theta_3,v_1,v_2,v_3,t_{0,1},t_{d,1},t_{0,2},t_{d,2},t_{0,3},t_{d,3})$$

约束条件:

1. 无人机速度范围：$70\le v_i\le 140, i\in\{1,2,3\}$.
2. 投弹时间范围：$t_{0,i}\ge 0$.
3. 引爆延迟范围：$t_{d,i}\ge 0$.
4. 烟幕弹飞行时间范围：$t_{fly,i}=t_{0,i}+t_{d,i}$，且 $\min t_{fly,i}\le t_{\text{M1_arrival}}=67.00s$

5.4.4 模型求解与计算

对于问题 4，采取和问题 3 类似的遗传算法进行优化，保留精英选择和局部搜索增强机制。遗传算法参数为:

- 迭代次数: 100,
- 种群数量: 300,
- 精英比例: 0.1,
- 交叉率: 0.7,
- 突变率: 0.4,

计算结果如下:

- **最大有效遮蔽时长**: **9.24s**
- **无人机 FY1 飞行参数**:
  - 飞行方向: 48.01 度
  - 飞行速度: 132.13 m/s
- **无人机 FY2 飞行参数**:
  - 飞行方向: 304.16 度
  - 飞行速度: 100.37 m/s
- **无人机 FY3 飞行参数**:
  - 飞行方向: 76.07 度
  - 飞行速度: 103.39 m/s
- **三枚烟幕弹投放参数**:
  - FY1 烟幕弹：投弹时间 0.10 s，引爆延迟 0.00 s。
  - FY2 烟幕弹：投弹时间 12.49 s，引爆延迟 4.29 s。
  - FY3 烟幕弹：投弹时间 30.45 s，引爆延迟 0.00 s。

最终结果已按要求保存至 **‘result2.xlsx‘ 文件**中。

5.5 问题五

5.5.1 运动学模型

导弹的运动轨迹、烟幕弹的运动轨迹均与问题二相同。

1. 导弹 M1,M2,M3 的运动轨迹导弹 Mi($i \in [1,2,3]$) 从初始位置 $\boldsymbol{P}_{Mi,0}$ 以 300m/s 的速度飞向原点 $\boldsymbol{P}_{Target} = (0,0,0)$ 。其单位方向向量为:

$$\boldsymbol{u}_{Mi} = \frac{\boldsymbol{P}_{Target} - \boldsymbol{P}_{Mi,0}}{||\boldsymbol{P}_{Target} - \boldsymbol{P}_{Mi,0}||}$$

导弹 Mi 在时刻 t 的位置 $\boldsymbol{P}_{Mi}(t)$ 为:

$$\boldsymbol{P}_{Mi}(t) = \boldsymbol{P}_{Mi,0} + \boldsymbol{u}_{Mi} \cdot V_{Mi} \cdot t$$

其中 $V_{Mi} = 300\text{m/s}$。导弹到达时间 $T_{Mi,arrival} = ||\boldsymbol{P}_{Mi,0}||/V_{Mi}$。

2. 无人机 FY1,FY2,FY3,FY4,FY5 的运动轨迹无人机 FYi($i \in [1,2,3,4,5]$) 从初始位置 $\boldsymbol{P}_{FYi,0}$ 等高度匀速直线飞行。设其水平飞行方向角为 θ_i，速度为 v_i。其速度向量为 $\boldsymbol{V}_{FYi} = (v_i \cos\theta_i, v_i \sin\theta_i, 0)$。无人机在时刻 t 的位置 $\boldsymbol{P}_{FYi}(t)$ 为:

$$\boldsymbol{P}_{FYi}(t) = \boldsymbol{P}_{FYi,0} + \boldsymbol{V}_{FYi} \cdot t$$

5.5.2 有效遮蔽判据

我们采用圆柱上下底面的离散点来代表真目标。在每个时刻 t，如果从某一导弹位置 $\boldsymbol{P}_{Mi}(t)$ 到真目标上任意一点 $\boldsymbol{P}_{target}$ 的视线都与此时刻区域内所有云团的并集相交，则该时刻是该导弹有效遮蔽时刻。

简化判据：若存在烟幕云团中心 $\boldsymbol{P}_{cloud}(t)$ 到线段 $\overline{\boldsymbol{P}_{Mi}(t)\boldsymbol{P}_{target}}$ 的最短距离小于等于烟幕云团半径 $R_{cloud} = 10\text{m}$，则该点被遮蔽。

若圆柱上下底面的所有离散点均被一个烟幕遮蔽，则将该时刻计入该烟幕的单独有效遮蔽时刻。

总的有效遮蔽时间设为三枚导弹的有效遮蔽时间之和。

5.5.3 优化模型

目标函数: 最大化总有效遮蔽时间 T_{eff}。

$$\max \quad T_{eff} = \sum_{k=1}^{3} \int_{t_{0,1}}^{max\{t_{boom,i}\}+20} I_k(\text{真目标完全被遮蔽})\, dt$$

其中 I_k 为指示函数，表示对于导弹 Mk，真目标完全被遮蔽。$max\{t_{boom,i}\} + 20s$ 为最后一枚烟雾弹失效的时间。

决策变量:

$$\boldsymbol{x} = (\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, v_1, v_2, v_3, v_4, v_5, t_{0,1,1}, t_{0,1,2}, t_{0,1,3}, t_{0,2,1}, \ldots, t_{0,5,3}, t_{d,1,1}, \ldots, t_{d,5,3})$$

约束条件:

1. 无人机速度范围：$70 \leq v_i \leq 140, i \in \{1, 2, 3\}$.
2. 投弹时间间隔：$t_{0,i,2} \geq t_{0,i,1} + 1.0$ 且 $t_{0,i,3} \geq t_{0,i,2} + 1.0$.
3. 投弹时间范围：$t_{0,i,j} \geq 0$.
4. 引爆延迟范围：$t_{d,i,j} \geq 0$.
5. 烟幕弹飞行时间范围：$t_{fly,i,j} = t_{0,i,j} + t_{d,i,j}$，且 $\min_{i,j} t_{fly,i,j} \leq \max_k t_{M_k,\text{arrival}} = 67.00s$

5.5.4 模型求解与计算

对于问题 5，采取和问题 3 类似的遗传算法进行优化，保留精英选择和局部搜索增强机制。为优化算法的运行时间，我们将变量整理为矩阵形式，并大量引入 Numpy 库中的方法[14]，实现了程序计算的加速。

由于问题的规模过大，对变量进行初始采样时极难采到遮蔽时间非 0 的点，导致遗传算法进展缓慢。经过对无效采样点的分析，我们发现大多采样点都是由于烟幕弹飞行时间过大而无效。因此，我们引入了缩放常数 $N_{scale} > 1$，将遗传算法中投弹时间、引爆延迟时间变量（即 $t_{0,1,1}, t_{0,1,2}, \ldots, t_{d,2,3}, \ldots, t_{d,5,3}$）的初始取值范围设为 $[0, \max_k t_{M_k,\text{arrival}}/N_{scale}]$，以更有效地找到算法中的优秀初值，进一步开始迭代。经蒙特卡洛方法试验，引入缩放常数之前，平均每 10000 次采样中只有 2 个遮蔽时间非 0 的变量组合；将缩放常数设为 10 时，平均每 10000 次采样有 35 个遮蔽时间非 0 的变量组合，效果良好。

遗传算法参数为：

- 迭代次数: 150,
- 种群数量: 1000,
- 精英比例: 0.01,
- 交叉率: 0.7,
- 突变率: 0.4,

用改进后的遗传算法，150 迭代次数下计算出的最优遮蔽时间为 10.35s，结果如下：

- **最大有效遮蔽时长**: **10.35 s**
- **无人机 FY1 飞行参数**:
  - 飞行方向: 6.6118 度
  - 飞行速度: 128.4002 m/s
- **无人机 FY2 飞行参数**:

  - 飞行方向: 0 度
  - 飞行速度: 94.6495 m/s
- **无人机 FY3 飞行参数**:
  - 飞行方向: 190.2068 度
  - 飞行速度: 105.7113 m/s
- **无人机 FY4 飞行参数**:
  - 飞行方向: 87.5345 度
  - 飞行速度: 100.3599 m/s
- **无人机 FY5 飞行参数**:
  - 飞行方向: 117.8391 度
  - 飞行速度: 140 m/s
- **烟幕弹投放参数**:
  - FY1 烟幕弹 1：投弹间隔 0 s，引爆延迟 0 s。
  - FY1 烟幕弹 2：投弹间隔 1 s，引爆延迟 0 s。
  - FY1 烟幕弹 3：投弹间隔 6.76 s，引爆延迟 0 s。
  - FY2 烟幕弹 1：投弹间隔 1.18 s，引爆延迟 7.26 s。
  - FY2 烟幕弹 2：投弹间隔 21.40 s，引爆延迟 16.66 s。
  - FY2 烟幕弹 3：投弹间隔 17.69 s，引爆延迟 0 s。
  - FY3 烟幕弹 1：投弹间隔 13.23 s，引爆延迟 27.25 s。
  - FY3 烟幕弹 2：投弹间隔 1 s，引爆延迟 22.85 s。
  - FY3 烟幕弹 3：投弹间隔 1 s，引爆延迟 15.12 s。
  - FY4 烟幕弹 1：投弹间隔 1.44 s，引爆延迟 34.74 s。
  - FY4 烟幕弹 2：投弹间隔 41.73 s，引爆延迟 0 s。
  - FY4 烟幕弹 3：投弹间隔 2.36 s，引爆延迟 3.67 s。
  - FY5 烟幕弹 1：投弹间隔 11.57 s，引爆延迟 1.46 s。
  - FY5 烟幕弹 2：投弹间隔 24.09 s，引爆延迟 22.38 s。
  - FY5 烟幕弹 3：投弹间隔 4.74 s，引爆延迟 0 s。

最终结果已按要求保存至 **‘result3.xlsx‘ 文件**中。

在 200 以上次迭代后最优时间为 12.60s，但程序运行尚未结束，无法给出具体的变量数值。由于在 200 次以上迭代过程中最优时间仍在不断稳定增加，我们相信在迭代次数足够多时，算法可以给出一个更令人满意的结果。

六、 模型的评价与推广

6.1 模型优点

6.1.1 问题一

在问题一中，我们创新性地提出了“视线锥”的描述，准确地分析了烟雾云团对目标圆柱实现有效遮蔽的情况，通过立体几何中的圆锥投影方法，准确而完备地计算云雾遮蔽原著的情况，其效果显著好于将目标圆柱简化为点或线。

6.1.2 问题二

在问题二中，我们采用粒子群优化算法，配合分层优化，既有效地提升了模型运算速度，同时又较好地避免了优化结果陷入局部最优的情况。

6.1.3 问题三至五

在这三个问题中，我们在与 PSO，CMA，DE 算法进行对比后，选择了遗传算法作为解决问题的基本框架，同时复合精英选择策略，有效避免搜索范围过大导致的不收敛问题，又保证了最终结果的全局最优效果。

6.2 模型不足

6.2.1 问题一

采用的“视线锥”模型，虽然准确，但如果在有限时间内，可能导致计算复杂，不易快速得出结果。

6.2.2 现实条件

虽考虑了诸多条件，但模型假设相对理想化，在实际运用中可能导致浮动偏差。

6.3 模型推广

该模型可推广应用于其他多目标优化场景的策略设计问题，如多外卖员配送路径问题、公共交通路线规划问题等，这些场景同样需要考虑有限时间空间内的最大化效益。

6.4 敏感性分析

从问题三开始，我们对算法找到的最优策略进行了敏感性分析，发现：

1. 苛刻的成功判据：模型要求全部 200 条关键视线必须同时被遮挡，才算作有效遮蔽。这是一个“全或无”的刚性标准，不存在“部分成功”。因此，哪怕环境的微小变化只导致了 1 条视线未能被挡住，在该时刻的遮蔽效果就被判定为彻底失败任何微小的扰动（如导弹速度快了 1%、烟幕下沉速度变了 2%）遮蔽时间瞬间从 6.6 秒跌落至极低, 有效解分布较为分散和稀疏。
2. 时空强耦合的物理过程：该问题中过多变量的耦合和复杂的约束条件

导致系统的敏感度较高。

参考文献

[1] 黄娜, 牛兴林, 骆正友, 等. 特种弹药中烟雾干扰弹的制造与性能优化研究[J]. 中国军转民, 2025(09): 22-23.

[2] 豆包. Doubao. 字节跳动, 2025–09–05.

[3] 王申怀. 圆锥曲线是椭圆双曲线和抛物线的解析证明[J/OL]. 数学通报, 2003(4): 9. DOI: 10.3969/j.issn.0583-1458.2003.04.003.

[4] 公茂果, 焦李成, 杨咚咚, 等. 进化多目标优化算法研究[J/OL]. 软件学报, 2009, 20(2): 271-289. DOI: 10.3724/SP.J.1001.2009.03483.

[5] 王树朋, 徐旺, 刘湘德, 等. 基于自适应遗传算法的多无人机协同任务分配[J/OL]. 电子信息对抗技术, 2021, 36(1): 59-64. DOI: 10.3969/j.issn.1674-2230.2021.01.013.

[6] 张正贵, 吴孝先. 圆锥面截交线投影特性的研究[J/OL]. 沈阳大学学报, 2002, 14(4): 31-32. DOI: 10.3969/j.issn.1008-9225.2002.04.009.

[7] ChatGPT. GPT-5. OpenAI, 2025–09–07.

[8] 蒋硕. PSO 算法改进及其在多无人机协同任务分配中的应用[D/OL]. 中国矿业大学, 2018. DOI: 10.27623/d.cnki.gzkyu.2018.000009.

[9] 杨维, 李歧强. 粒子群优化算法综述[J/OL]. 中国工程科学, 2004, 6(5): 87-94. DOI: 10.3969/j.issn.1009-1742.2004.05.018.

[10] Gemini. Gemini 2.5 Pro. Google DeepMind, 2025–09–07.

[11] 朱陆陆. 蒙特卡洛方法及应用[D]. 华中师范大学, 2014.

[12] 徐磊. 基于遗传算法的多目标优化问题的研究与应用[D]. 中南大学, 2007.

[13] 祁荣宾, 钱锋, 杜文莉, 等. 基于精英选择和个体迁移的多目标遗传算法[J/OL]. 控制与决策, 2007(02): 164-168. DOI: 10.13195/j.cd.2007.02.46.qirb.009.

[14] DeepSeek. DeepSeek-R1. 深度求索（DeepSeek）, 2025–09–06.

附录 A　支撑材料列表

文件名	说明
AI 工具使用详情	本文使用 AI 工具的汇总及关键交互图
result1.xlsx	第三问结果
result2.xlsx	第四问结果
result3.xlsx	第五问结果
T0_initial_animation.m	初始状态绘图 matlab 源码
T1_motion_tracker.m	第一问路径绘图源码
T1_points.py	第一问离散点建模源码
T1_real.py	第一问建模源码
T1_visualize.m	第一问可视化 matlab 源码
T2_MC.py	第二问蒙特卡洛求解源码
T2_PSO.py	第二问粒子群求解源码
T2_visualize.m	第二问可视化源码
T3_CMA.py	第三问 CMA 求解源码
T3_DE.py	第三问 DE 求解源码
T3_GA.py	第三问 GA 求解源码
T3_PSO.py	第三问 PSO 求解源码
T3_visualize_local_best.py	T3 局部最优解绘图源码
T3_visualize_process.py	T3 全过程模拟绘图源码
T4.py	T4 求解源码
T5.py	T5 求解源码

附录 B　问题一：圆柱体建模计算源码

```
import sympy as sp
import numpy as np
from scipy.optimize import minimize
from tqdm import tqdm

G = -9.8

def distance_point_to_line_segment(p, a, b):
    """计算点p到线段ab的最短距离"""
    if np.all(a == b): return np.linalg.norm(p - a)
    ap, ab = p - a, b - a
    t = np.dot(ap, ab) / np.dot(ab, ab)
    if t < 0.0: return np.linalg.norm(p - a)
    if t > 1.0: return np.linalg.norm(p - b)
    return np.linalg.norm(p - (a + t * ab))

def cone_surface_equation(vertex:np.ndarray, ball_center:np.ndarray, radius:float):
    """生成圆锥面方程的符号表达式"""
    x, y, z = sp.symbols('x y z')
    pointP = np.array([x,y,z])

    VC = ball_center - vertex
    VP = pointP - vertex

    # VP的模方
    VP_sq = np.dot(VP, VP)
    # VC的模方
    VC_sq = np.dot(VC, VC)
    # VC点乘VP
    VCdotVP = np.dot(VC, VP)
    # 构造圆锥面方程
    equation = VCdotVP**2 - (VC_sq - radius**2)*VP_sq
    return sp.simplify(equation)

def find_intersection_curve(cone_eq, plane_eq, output=False):
    """求解圆锥面与平面的交线"""
    x, y, z = sp.symbols('x y z')

    # 从平面方程解出一个变量
    if plane_eq.has(z):
        z_expr = sp.solve(plane_eq, z)[0]
        # 代入圆锥面方程
        projection_eq = cone_eq.subs(z, z_expr)
        projection_eq = sp.simplify(projection_eq)
```

```
        if output:
            print("\n交线在xy平面的投影方程:")
            print(projection_eq, "= 0")

        return projection_eq

    return None

def min_distance_to_curve(x0, y0, eq, num_initial_points=8, tol=1e-6):
    "用SLSQP计算点到曲线的最短距离"
    x, y = sp.symbols('x y')
    ellipse_func = sp.lambdify((x, y), eq, 'numpy')

    def objective(params):
        return (params[0] - x0)**2 + (params[1] - y0)**2

    def constraint(params):
        return ellipse_func(params[0], params[1])

    cons = {'type': 'eq', 'fun': constraint}

    # 求解椭圆中心
    center_sols = sp.solve([eq.diff(x), eq.diff(y)], (x, y))
    if not center_sols:
        raise ValueError("无法求解椭圆中心")
    h, k = center_sols[x], center_sols[y]

    initial_points = []
    for i in range(num_initial_points):
        angle = 2 * np.pi * i / num_initial_points # 覆盖完整圆周
        m = np.tan(angle)
        line_eq = y - k - m * (x - h)

        # 求解直线与椭圆的交点
        solutions = sp.solve([line_eq, eq], (x, y))

        for s in solutions:
            try:
                # 显式转换为数值并过滤实数解
                x_val = float(s[0].evalf())
                y_val = float(s[1].evalf())
                if np.isreal(x_val) and np.isreal(y_val):
                    initial_points.append([x_val, y_val])
            except (TypeError, ValueError):
                continue # 跳过无效解

    # 优化每个初始点
```

```
    min_distance = float('inf')
    for point in initial_points:
        result = minimize(
            objective, point,
            constraints=cons,
            method='SLSQP',
            tol=tol
        )

        distance = np.sqrt(result.fun)
        if distance < min_distance:
            min_distance = distance

    return min_distance

def target_in_shadow(missile_location:np.ndarray, bomb_location:np.ndarray,
     target_location:tuple):
    """判断真目标圆柱是否在投影范围内"""

    reason = '无'

    if np.linalg.norm(missile_location - bomb_location) <= 10:
        # 导弹在烟雾范围内
        return True, reason
    else:
        # 计算投影圆锥面方程
        x, y, z = sp.symbols('x y z')
        cone_eq = cone_surface_equation(vertex=missile_location, ball_center=bomb_location,
             radius=10)

        x0 = target_location[0]
        y0 = target_location[1]

        if missile_location[2] > bomb_location[2] + 10:
            # 导弹位置在烟雾上方，此时圆锥面与水平面交线为椭圆
            for z0 in [0, 10]:
                # 判断圆柱上下底面是否在投影内
                plane_eq = z - z0 # 水平面
                ellipse_eq = find_intersection_curve(cone_eq, plane_eq) # 求解交线
                ellipse_func = sp.lambdify((x, y), ellipse_eq, 'numpy')
                center = sp.solve([ellipse_eq.diff(x),ellipse_eq.diff(y)],(x,y))
                f_center = ellipse_func(center[x], center[y])
                f_x0y0 = ellipse_func(x0, y0)

                if f_center * f_x0y0 < 0:
                    # 圆心在椭圆外部
                    reason = f'圆心在椭圆外部,{z0}'
```

```
                return False, reason
            else:
                # 圆心在椭圆内部，计算圆心到椭圆的最小距离
                min_d = min_distance_to_curve(x0, y0, ellipse_eq)
                if min_d < 7:
                    reason = f'圆心在椭圆内部,{z0}'
                    return False, reason
        else:
            # 导弹位置和烟雾重合，此时圆锥面与水平面交线为双曲线，需要考虑双曲线的一支
            # 判断圆柱下底面是否在投影内
            plane_eq = z # 水平面
            ellipse_eq = find_intersection_curve(cone_eq, plane_eq) # 求解交线
            ellipse_func = sp.lambdify((x, y), ellipse_eq, 'numpy')
            center = sp.solve([ellipse_eq.diff(x),ellipse_eq.diff(y)],(x,y))
            f_center = ellipse_func(center[x], center[y])
            f_x0y0 = ellipse_func(x0, y0)

            if f_center * f_x0y0 > 0:
                # 圆心在双曲线内侧
                reason = '圆心在双曲线内侧'
                return False, reason
            else:
                # 圆心在双曲线外侧，计算圆心到双曲线的最小距离
                min_d = min_distance_to_curve(x0, y0, ellipse_eq)
                if min_d < 7:
                    reason = '圆心在双曲线外侧'
                    return False, reason

        return True, reason

def get_cover_time_real(particle:np.ndarray, missile_flying_time, output=False):
    """
    通过求解圆锥面和水平面交线，计算给定无人机方向、角度、飞行时间和起爆时间时的目标被遮蔽的总时间
    :param particle: 含有参数的粒子，[drone_speed, drone_angle, drone_flying_time,
        bomb_burst_time]
    :param missile_flying_time: 导弹飞行总时间
    :param output:控制是否输出
    :return: 目标被遮蔽总时间
    """
    drone_speed, drone_angle, drone_flying_time, bomb_burst_time = particle.tolist()
    # 无人机方向、角度、飞行时间、起爆时间

    # 若无人机飞行时间+起爆时间>=导弹飞行时间，遮蔽时间为0
    if drone_flying_time + bomb_burst_time >= missile_flying_time:
        return 0

    fake_target_location = np.array([0,0,0]) # 假目标位置
```

```
real_target_location_points = [] # 取样的真目标点
for z0 in [0,10]:
    for i in range(0, 100):
        real_target_location_points.append(np.array([7*np.cos(i/50*np.pi),200+7*np.sin(i/50*np.pi),z0]))
drone_start_location = np.array([17800,0,1800]) # 无人机起始位置

drone_direction = np.array([np.cos(drone_angle),np.sin(drone_angle),0]) # 无人机方向向量

bomb_start_location = drone_start_location + drone_direction * drone_speed *
    drone_flying_time # 烟雾弹落下位置

bomb_burst_location_x = bomb_start_location[0] + drone_direction[0] * drone_speed *
    bomb_burst_time
bomb_burst_location_y = bomb_start_location[1] + drone_direction[1] * drone_speed *
    bomb_burst_time
bomb_burst_location_z = bomb_start_location[2] + G * bomb_burst_time**2 / 2
bomb_burst_location = np.array([bomb_burst_location_x, bomb_burst_location_y,
    bomb_burst_location_z]) # 烟雾弹爆炸位置

M1_start_location = np.array([20000,0,2000]) # 导弹起始位置
M1_speed = 300 # 导弹速度
M1_direction = (fake_target_location -
    M1_start_location)/np.linalg.norm(fake_target_location - M1_start_location) #
    导弹方向向量

M1_count_location = M1_start_location + M1_direction * (drone_flying_time +
    bomb_burst_time) * M1_speed # 烟雾弹爆炸时导弹位置

if output:
    print(f'烟雾弹抛下位置:{bomb_start_location}')
    print(f'烟雾弹爆炸位置:{bomb_burst_location}')
    print(f'导弹方向:{M1_direction}')
    print(f'烟雾弹爆炸时导弹位置:{M1_count_location}')

bomb_speed = 3
bomb_direction = np.array([0,0,-1])
bomb_location = bomb_burst_location.copy()
M1_location = M1_count_location.copy()
cover_time = 0
dt = 0.1
cover = False

# 模拟运动过程
for t in tqdm(np.arange(0, 20, dt),bar_format='模拟进度: {l_bar}{bar}|
    [{elapsed}<{remaining}, {rate_fmt}{postfix}]'):

    res = target_in_shadow(M1_location, bomb_location, (0,200))
```

```
        if res[0] and not cover:
            cover = True
            start_time = t + dt # 记录开始遮蔽时间
        if not res[0] and cover:
            cover = False
            cover_time += t - start_time # 记录结束遮蔽时间
            # print(res[1])

        # 更新烟雾、导弹位置
        bomb_location += dt * bomb_speed * bomb_direction
        M1_location += dt * M1_speed * M1_direction

        if M1_location[2] < bomb_location[2] - 10 or t >= 20:
            # 导弹位置低于烟雾位置或时间超过爆炸20s后结束计算
            if cover:
                cover_time += t - start_time # 结束时，记录遮蔽时间
            break

    return cover_time

if __name__ == '__main__':

    particle = np.array([120, np.pi, 1.5, 3.6]) # 无人机数据

    M1_start_location = np.array([20000,0,2000])
    M1_speed = 300
    missile_flying_time = np.linalg.norm(M1_start_location)/M1_speed # 导弹飞行时间

    # 精确情况计算
    print('\n计算精确情况:')
    real_cover_time = get_cover_time_real(particle, missile_flying_time, output=True)
    print(f'精确计算的导弹被遮蔽时间为:{real_cover_time:.3f} s')
```

附录 C　问题一：离散点建模计算源码

```
import numpy as np

G = -9.8

def distance_point_to_line_segment(p, a, b):
    """计算点p到线段ab的最短距离"""
    if np.all(a == b): return np.linalg.norm(p - a)
    ap, ab = p - a, b - a
    t = np.dot(ap, ab) / np.dot(ab, ab)
    if t < 0.0: return np.linalg.norm(p - a)
```

```
    if t > 1.0: return np.linalg.norm(p - b)
    return np.linalg.norm(p - (a + t * ab))

def get_cover_time_points(particle:np.ndarray, missile_flying_time, output=False):
    """
    通过取样真目标上的点，计算给定无人机方向、角度、飞行时间和起爆时间时的目标被遮蔽的总时间
    :param particle: 含有参数的粒子，[drone_speed, drone_angle, drone_flying_time,
        bomb_burst_time]
    :param missile_flying_time: 导弹飞行总时间
    :param output:控制是否输出
    :return: 目标被遮蔽总时间
    """
    drone_speed, drone_angle, drone_flying_time, bomb_burst_time = particle.tolist()
    # 无人机方向、角度、飞行时间、起爆时间

    # 若无人机飞行时间+起爆时间>=导弹飞行时间，遮蔽时间为0
    if drone_flying_time + bomb_burst_time >= missile_flying_time:
        return 0

    fake_target_location = np.array([0,0,0]) # 假目标位置
    real_target_location_points = [] # 取样的真目标点
    for z0 in [0,10]:
        for i in range(0, 100):
            real_target_location_points.append(np.array([7*np.cos(i/50*np.pi),200+7*np.sin(i/50*np.pi),z0]))
    drone_start_location = np.array([17800,0,1800]) # 无人机起始位置

    drone_direction = np.array([np.cos(drone_angle),np.sin(drone_angle),0]) # 无人机方向向量

    bomb_start_location = drone_start_location + drone_direction * drone_speed *
        drone_flying_time # 烟雾弹落下位置

    bomb_burst_location_x = bomb_start_location[0] + drone_direction[0] * drone_speed *
        bomb_burst_time
    bomb_burst_location_y = bomb_start_location[1] + drone_direction[1] * drone_speed *
        bomb_burst_time
    bomb_burst_location_z = bomb_start_location[2] + G * bomb_burst_time**2 / 2
    bomb_burst_location = np.array([bomb_burst_location_x, bomb_burst_location_y,
        bomb_burst_location_z]) # 烟雾弹爆炸位置

    M1_start_location = np.array([20000,0,2000]) # 导弹起始位置
    M1_speed = 300 # 导弹速度
    M1_direction = (fake_target_location -
        M1_start_location)/np.linalg.norm(fake_target_location - M1_start_location) #
        导弹方向向量

    M1_count_location = M1_start_location + M1_direction * (drone_flying_time +
        bomb_burst_time) * M1_speed # 烟雾弹爆炸时导弹位置
```

```
    if output:
        print(f'烟雾弹抛下位置:{bomb_start_location}')
        print(f'烟雾弹爆炸位置:{bomb_burst_location}')
        print(f'导弹方向:{M1_direction}')
        print(f'烟雾弹爆炸时导弹位置:{M1_count_location}')

    cover = False
    t = 0
    delta_t = 0.005 # 时间步长
    bomb_speed = 3 # 烟雾下落速度
    bomb_direction = np.array([0,0,-1]) # 烟雾下落方向向量
    bomb_location = bomb_burst_location.copy()
    M1_location = M1_count_location.copy()
    cover_time = 0
    while(1):

        bomb_location += delta_t * bomb_speed * bomb_direction
        M1_location += delta_t * M1_speed * M1_direction

        if M1_location[2] < bomb_location[2]- 10 or t >= 20:
            # 导弹位置低于烟雾位置或时间超过爆炸20s后结束计算
            if cover:
                end_time = t
                cover_time += end_time - start_time
            break

        all_in = True
        for point in real_target_location_points:
            distance = distance_point_to_line_segment(bomb_location, M1_location, point) #
                导弹与真目标连线到烟雾中心距离
            if distance > 10:
                all_in = False

        if not cover and all_in:
            cover = True
            start_time = t
        if cover and not all_in:
            cover = False
            end_time = t
            cover_time += end_time - start_time
        t += delta_t
    return cover_time

if __name__ == '__main__':

    particle = np.array([120, np.pi, 1.5, 3.6]) # 变量取值
```

```
    M1_start_location = np.array([20000,0,2000])
    M1_speed = 300
    missile_flying_time = np.linalg.norm(M1_start_location)/M1_speed # 导弹飞行时间

        # 取样点近似情况计算
    print('\n计算近似情况:')
    points_cover_time = get_cover_time_points(particle,missile_flying_time=67,output=True)
    print(f'取样点近似计算的导弹被遮蔽时间为:{points_cover_time:.3f} s')
```

附录 D　问题二：PSO 算法求解源码

```
     import numpy as np
from tqdm import tqdm
G = -9.8

def distance_point_to_line_segment(p, a, b):
    """计算点p到线段ab的最短距离"""
    if np.all(a == b): return np.linalg.norm(p - a)
    ap, ab = p - a, b - a
    t = np.dot(ap, ab) / np.dot(ab, ab)
    if t < 0.0: return np.linalg.norm(p - a)
    if t > 1.0: return np.linalg.norm(p - b)
    return np.linalg.norm(p - (a + t * ab))

def get_cover_time(particle:np.ndarray, missile_flying_time, output=False):
    """
    用取样点近似计算给定无人机方向、角度、飞行时间和起爆时间时的目标被遮蔽的总时间
    :param particle: 含有参数的粒子, [drone_speed, drone_angle, drone_flying_time,
        bomb_burst_time]
    :param missile_flying_time: 导弹飞行总时间
    :param output:控制是否输出
    :return: 目标被遮蔽总时间
    """
    drone_speed, drone_angle, drone_flying_time, bomb_burst_time = particle.tolist()
    # 无人机方向、角度、飞行时间、起爆时间

    # 若无人机飞行时间+起爆时间>=导弹飞行时间, 遮蔽时间为0
    if drone_flying_time + bomb_burst_time >= missile_flying_time:
        return 0

    fake_target_location = np.array([0,0,0]) # 假目标位置
    real_target_location_points = []
    for z0 in [0,10]:
        for i in range(0, 100):
```

```
        real_target_location_points.append(np.array([7*np.cos(i/50*np.pi),200+7*np.sin(i/50*np.pi),z0]))
drone_start_location = np.array([17800,0,1800]) # 无人机起始位置

drone_direction = np.array([np.cos(drone_angle),np.sin(drone_angle),0]) # 无人机方向向量

bomb_start_location = drone_start_location + drone_direction * drone_speed *
    drone_flying_time # 烟雾弹落下位置

bomb_burst_location_x = bomb_start_location[0] + drone_direction[0] * drone_speed *
    bomb_burst_time
bomb_burst_location_y = bomb_start_location[1] + drone_direction[1] * drone_speed *
    bomb_burst_time
bomb_burst_location_z = bomb_start_location[2] + G * bomb_burst_time**2 / 2
bomb_burst_location = np.array([bomb_burst_location_x, bomb_burst_location_y,
    bomb_burst_location_z]) # 烟雾弹爆炸位置

M1_start_location = np.array([20000,0,2000]) # 导弹起始位置
M1_speed = 300 # 导弹速度
M1_direction = (fake_target_location -
    M1_start_location)/np.linalg.norm(fake_target_location - M1_start_location) #
    导弹方向向量

M1_count_location = M1_start_location + M1_direction * (drone_flying_time +
    bomb_burst_time) * M1_speed # 烟雾弹爆炸时导弹位置

if output:
    print(f'烟雾弹抛下位置:{bomb_start_location}')
    print(f'烟雾弹爆炸位置:{bomb_burst_location}')
    print(f'导弹方向:{M1_direction}')
    print(f'烟雾弹爆炸时导弹位置:{M1_count_location}')

cover = False
t = 0
delta_t = 0.1 # 时间步长
bomb_speed = 3 # 烟雾下落速度
bomb_direction = np.array([0,0,-1]) # 烟雾下落方向向量
bomb_location = bomb_burst_location.copy()
M1_location = M1_count_location.copy()
cover_time = 0
while(1):

    bomb_location += delta_t * bomb_speed * bomb_direction
    M1_location += delta_t * M1_speed * M1_direction

    if M1_location[2] < bomb_location[2]- 10 or t >= 20:
        # 导弹位置低于烟雾位置或时间超过爆炸20s后结束计算
        if cover:
```

```
            end_time = t
            cover_time += end_time - start_time
        break

    all_in = True
    for point in real_target_location_points:
        distance = distance_point_to_line_segment(bomb_location, M1_location, point) #
            导弹与真目标连线到烟雾中心距离
        if distance > 10:
            all_in = False

    if not cover and all_in:
        cover = True
        start_time = t
    if cover and not all_in:
        cover = False
        end_time = t
        cover_time += end_time - start_time
    t += delta_t

if output:
    print(f'开始遮蔽时间:{start_time}')
    print(f'结束遮蔽时间:{end_time}')
return cover_time

def pso_optimizer(bounds, missile_flying_time, num_particles=100, max_iter=100, w=0.8, c1=1.5,
    c2=1.5):
    """
    粒子群优化算法
    :param bounds: 变量边界列表，格式为[(min, max), (min, max), (min, max), (min, max)]
    :param missile_flying_time: 导弹飞行总时间
    :param num_particles: 粒子数量
    :param max_iter: 最大迭代次数
    :param w: 惯性权重
    :param c1: 个体学习因子
    :param c2: 群体学习因子
    :return: (全局最佳位置, 全局最佳适应值)
    """
    # 变量维度
    dimensions = len(bounds)

    # 初始化粒子群
    particles = np.random.uniform(low=[b[0] for b in bounds],
                                  high=[b[1] for b in bounds],
                                  size=(num_particles, dimensions))

    # 初始化粒子速度
```

```
velocities = np.zeros((num_particles, dimensions))

# 初始化个体最佳位置和适应值
personal_best_positions = particles.copy()
personal_best_values_list = []
for i in tqdm(range(particles.shape[0]),bar_format='初始化进度: {l_bar}{bar}|
    {n_fmt}/{total_fmt} [{elapsed}<{remaining}, {rate_fmt}{postfix}]'):
    p = particles[i]
    personal_best_values_list.append(get_cover_time(p,
        missile_flying_time=missile_flying_time))
personal_best_values = np.array(personal_best_values_list)

# 寻找初始全局最优
global_best_idx = np.argmin(personal_best_values)
global_best_position = personal_best_positions[global_best_idx]
global_best_value = personal_best_values[global_best_idx]

# 迭代优化
for _ in tqdm(range(max_iter),bar_format='优化进度: {l_bar}{bar}| {n_fmt}/{total_fmt}
    [{elapsed}<{remaining}, {rate_fmt}{postfix}]'):
    for i in range(num_particles):
        # 计算当前粒子适应值
        current_value = get_cover_time(particles[i],missile_flying_time=missile_flying_time)

        # 更新个体最优
        if current_value > personal_best_values[i]:
            personal_best_positions[i] = particles[i].copy()
            personal_best_values[i] = current_value

            # 更新全局最优
            if current_value > global_best_value:
                global_best_position = particles[i].copy()
                global_best_value = current_value

    # 更新速度和位置
    for i in range(num_particles):
        r1, r2 = np.random.random(), np.random.random()
        velocities[i] = (w * velocities[i] +
                        c1 * r1 * (personal_best_positions[i] - particles[i]) +
                        c2 * r2 * (global_best_position - particles[i]))

        # 更新位置并确保在边界内
        particles[i] += velocities[i]
        for d in range(dimensions):
            low, high = bounds[d]
            particles[i, d] = np.clip(particles[i, d], low, high)
```

```python
    return global_best_position, global_best_value

if __name__ == '__main__':

    # 计算导弹飞行时间
    M1_start_location = np.array([20000,0,2000])
    M1_speed = 300
    missile_flying_time = np.linalg.norm(M1_start_location)/M1_speed

    # 应用PSO算法
    # 变量边界，顺序为：drone_speed, drone_angle, drone_flying_time, bomb_burst_time
    bounds = [(70,140),(0,np.pi),(0,missile_flying_time),(0,missile_flying_time)]

    print('正在应用PSO算法...')

    global_best_position, global_best_value = pso_optimizer(
        bounds=bounds,
        missile_flying_time=missile_flying_time
        )

    print(f'PSO算法找到的最优变量为:{global_best_position}')
    print(f'此时遮蔽时间为:{global_best_value:.3f}')
```

附录 E　问题三：遗传算法求解源码

```python
import numpy as np
import random

# 定义常量
FY1_START = np.array([17800, 0, 1800])
M1_START = np.array([20000, 0, 2000])
MISSILE_TARGET_POINT = np.array([0, 0, 0])
TARGET_CYLINDER_BASE_CENTER = np.array([0, 200, 0])
TARGET_CYLINDER_RADIUS = 7.0
TARGET_CYLINDER_HEIGHT = 10.0
MISSILE_SPEED = 300.0
CLOUD_FALL_SPEED = 3.0
CLOUD_RADIUS = 10.0
CLOUD_EFFECTIVE_DURATION = 20.0
G = 9.8

# 计算导弹飞行时间
M1_start_location = np.array([20000,0,2000])
M1_speed = 300
M1_ARRIVAL_TIME = np.linalg.norm(M1_start_location)/M1_speed
```

```python
# 生成目标圆柱体点集
num_target_points_per_face = 100
angles = np.linspace(0, 2 * np.pi, num_target_points_per_face, endpoint=False)
x_rim = TARGET_CYLINDER_BASE_CENTER[0] + TARGET_CYLINDER_RADIUS * np.cos(angles)
y_rim = TARGET_CYLINDER_BASE_CENTER[1] + TARGET_CYLINDER_RADIUS * np.sin(angles)
points_bottom = np.vstack([x_rim, y_rim, np.full_like(x_rim, 0)]).T
points_top = np.vstack([x_rim, y_rim, np.full_like(x_rim, TARGET_CYLINDER_HEIGHT)]).T
TARGET_POINTS = np.vstack([points_bottom, points_top])

# 运动过程模拟相关函数

def distance_point_to_line_segment(p, a, b):
    """计算点p到线段ab的最短距离"""
    if np.all(a == b):
        return np.linalg.norm(p - a)
    ap, ab = p - a, b - a
    norm_ab_sq = np.dot(ab, ab)
    if norm_ab_sq == 0:
        return np.linalg.norm(p - a)
    t = np.dot(ap, ab) / norm_ab_sq
    if t < 0.0:
        return np.linalg.norm(p - a)
    if t > 1.0:
        return np.linalg.norm(p - b)
    return np.linalg.norm(p - (a + t * ab))

def get_cover_time(particle, output=False):
    """
    用取样点近似计算目标被遮蔽的总时间
    :param particle: 参数粒子 [drone_speed, drone_angle, drone_flying_time,
                bomb_burst_time_1, rest_1, bomb_burst_time_2, rest_2, bomb_burst_time_3]
    :param missile_flying_time: 导弹飞行总时间
    :param output: 是否输出详细信息
    :return: 目标被遮蔽总时间
    """
    # 解析参数
    drone_speed = particle[0]
    drone_angle = particle[1]
    drone_flying_time = particle[2]
    bomb_burst_time_1 = particle[3]
    rest_1 = particle[4]
    bomb_burst_time_2 = particle[5]
    rest_2 = particle[6]
    bomb_burst_time_3 = particle[7]

    # 计算烟雾弹爆炸时间
```

```python
bomb_detonation_times = [
    drone_flying_time + bomb_burst_time_1,
    drone_flying_time + rest_1 + bomb_burst_time_2,
    drone_flying_time + rest_1 + rest_2 + bomb_burst_time_3
]

# 检查是否所有烟雾弹都在导弹到达后爆炸
if all(t >= M1_ARRIVAL_TIME for t in bomb_detonation_times):
    return 0

# 计算导弹方向向量
missile_direction = (MISSILE_TARGET_POINT - M1_START) / np.linalg.norm(MISSILE_TARGET_POINT
     - M1_START)

# 计算无人机方向向量
drone_direction = np.array([np.cos(drone_angle), np.sin(drone_angle), 0])

# 计算烟雾弹投放和爆炸位置
bomb_data = []
for i in range(3):
    if i == 0:
        drop_time = drone_flying_time
        burst_time = bomb_burst_time_1
    elif i == 1:
        drop_time = drone_flying_time + rest_1
        burst_time = bomb_burst_time_2
    else:
        drop_time = drone_flying_time + rest_1 + rest_2
        burst_time = bomb_burst_time_3

    # 投放位置
    drop_pos = FY1_START + drone_direction * drone_speed * drop_time

    # 爆炸位置（考虑重力影响）
    detonation_pos = drop_pos + drone_direction * drone_speed * burst_time + 0.5 *
         np.array([0, 0, -G]) * burst_time**2

    bomb_data.append({
        'bomb_id': i,
        'detonation_time': bomb_detonation_times[i],
        'expiry_time': bomb_detonation_times[i] + CLOUD_EFFECTIVE_DURATION,
        'detonation_pos': detonation_pos,
        'drop_pos': drop_pos
    })

    if output:
        print(f"第{i+1}颗烟雾弹投放位置: {drop_pos}")
```

```
        print(f"烟雾弹爆炸位置: {detonation_pos}")

# 时间步长和总遮蔽时间
delta_t = 0.2
cover_time = 0
cover_time_single = [0] * 3

# 模拟导弹飞行过程
for t in np.arange(0, M1_ARRIVAL_TIME, delta_t):
    missile_pos = M1_START + missile_direction * MISSILE_SPEED * t

    # 检查当前活跃的烟雾弹
    active_bombs = [
        bomb for bomb in bomb_data
        if bomb['detonation_time'] <= t <= bomb['expiry_time']
    ]

    if not active_bombs:
        continue

    # 检查所有目标点是否被遮蔽
    all_points_covered = True
    for point in TARGET_POINTS:
        point_covered = False

        # 检查该点是否被任何活跃烟雾弹遮蔽
        for bomb in active_bombs:
            # 计算烟雾当前位置（考虑下落）
            cloud_center = bomb['detonation_pos'] - np.array([0, 0, CLOUD_FALL_SPEED * (t -
                bomb['detonation_time'])])

            # 计算点到导弹-目标连线的距离
            distance = distance_point_to_line_segment(cloud_center, missile_pos, point)

            if distance <= CLOUD_RADIUS:
                point_covered = True
                break

        if not point_covered:
            all_points_covered = False
            break

    # 如果所有目标点都被遮蔽，增加遮蔽时间
    if all_points_covered:
        cover_time += delta_t

    # 计算单颗烟雾弹的遮蔽时间
```

```
        for bomb in active_bombs:
            # 计算烟雾当前位置
            cloud_center = bomb['detonation_pos'] - np.array([0, 0, CLOUD_FALL_SPEED * (t -
                bomb['detonation_time'])])
            all_points_covered_by_single_bomb = True

            for point in TARGET_POINTS:
                distance = distance_point_to_line_segment(cloud_center, missile_pos, point)
                if distance > CLOUD_RADIUS:
                    all_points_covered_by_single_bomb = False

            if all_points_covered_by_single_bomb:
                cover_time_single[bomb['bomb_id']] += delta_t

    if output:
        for i in range(3):
            print(f"第{i+1}颗烟雾弹遮蔽时间: {cover_time_single[i]:.2f} s")
        print(f"总遮蔽时间: {cover_time:.2f} s")

    return cover_time

# 遗传算法相关函数

def initialize_population(pop_size, bounds):
    """初始化种群"""
    population = np.zeros((pop_size, len(bounds)))
    for i in range(pop_size):
        for j in range(len(bounds)):
            population[i, j] = random.uniform(bounds[j][0], bounds[j][1])
    return population

def roulette_wheel_selection(population, fitness_scores):
    """轮盘赌选择"""
    total_fitness = sum(fitness_scores)
    if total_fitness == 0: # 避免除零错误
        return random.choice(population)
    selection_probs = [f / total_fitness for f in fitness_scores]
    return population[np.random.choice(len(population), p=selection_probs)]

def uniform_crossover(parent1, parent2):
    """均匀交叉"""
    child = parent1.copy()
    for i in range(len(parent1)):
        if random.random() < 0.5:
            child[i] = parent2[i]
    return child
```

```python
def gaussian_mutation(individual, bounds, mutation_prob, mutation_scale=0.1):
    """高斯变异"""
    mutated_individual = individual.copy()
    for i in range(len(individual)):
        if random.random() < mutation_prob:
            range_width = bounds[i][1] - bounds[i][0]
            mutation = np.random.normal(0, range_width * mutation_scale)
            mutated_individual[i] += mutation
            # 确保仍在边界内
            mutated_individual[i] = np.clip(mutated_individual[i], bounds[i][0], bounds[i][1])
    return mutated_individual

def local_search(individual, bounds):
    """对单个精英个体进行局部搜索"""
    best_neighbor = individual.copy()
    best_fitness = get_cover_time(best_neighbor)

    # 在小邻域内随机扰动20次
    for _ in range(20):
        neighbor = best_neighbor.copy()
        for j in range(2, 8): # 微调 t0i 和 dti
            range_width = bounds[j][1] - bounds[j][0]
            perturbation = np.random.normal(0, range_width * 0.02) # 更小的扰动
            neighbor[j] += perturbation

        for d in range(len(neighbor)):
            neighbor[d] = np.clip(neighbor[d], bounds[d][0], bounds[d][1])

        # 检查约束
        t01, t02, t03 = neighbor[2], neighbor[4], neighbor[6]
        dt3 = neighbor[7]
        if not (t02 >= t01 + 1.0 and t03 >= t02 + 1.0) or (t03 + dt3 + 20) > M1_ARRIVAL_TIME:
            continue

        neighbor_fitness = get_cover_time(neighbor)
        if neighbor_fitness > best_fitness:
            best_fitness = neighbor_fitness
            best_neighbor = neighbor

    return best_neighbor, best_fitness

def optimize_with_genetic():
    """遗传算法主函数"""
    # 定义决策变量的边界
    varbound = [
        (0, 2*np.pi),
        (70.0, 140.0),
```

```
    (0, M1_ARRIVAL_TIME-2.1), (0, M1_ARRIVAL_TIME),
    (1.0, M1_ARRIVAL_TIME-1.1), (0, M1_ARRIVAL_TIME),
    (2.0, M1_ARRIVAL_TIME), (0, M1_ARRIVAL_TIME)
]

# 配置遗传算法参数
GA_PARAMS = {
    'num_generations': 100,
    'population_size': 200,
    'elit_ratio': 0.1, # 精英比例
    'crossover_prob': 0.6,
    'mutation_prob': 0.4,
}

# 初始化
population = initialize_population(GA_PARAMS['population_size'], varbound)
best_solution_so_far = None
best_fitness_so_far = -1

# 迭代进化
for gen in range(GA_PARAMS['num_generations']):

    # 计算适应度
    fitness_scores = np.array([get_cover_time(ind) for ind in population])

    # 寻找当前代的精英
    num_elites = int(GA_PARAMS['population_size'] * GA_PARAMS['elit_ratio'])
    elite_indices = np.argsort(fitness_scores)[-num_elites:]
    elites = population[elite_indices]

    # 局部搜索增强
    print(f"第 {gen+1}/{GA_PARAMS['num_generations']} 代, 最佳时长:
        {np.max(fitness_scores):.4f}s. 正在微调 {num_elites} 个精英...")
    refined_elites = []
    for elite in elites:
        refined_sol, refined_fit = local_search(elite, varbound)
        refined_elites.append(refined_sol)

    # 如果局部搜索找到了全局更优解, 则更新
    current_gen_best_idx = np.argmax(fitness_scores)
    if fitness_scores[current_gen_best_idx] > best_fitness_so_far:
        best_fitness_so_far = fitness_scores[current_gen_best_idx]
        best_solution_so_far = population[current_gen_best_idx].copy()

    # 生成新一代
    new_population = list(refined_elites) # 精英直接进入下一代
```

```
        while len(new_population) < GA_PARAMS['population_size']:
            parent1 = roulette_wheel_selection(population, fitness_scores)
            parent2 = roulette_wheel_selection(population, fitness_scores)

            child = uniform_crossover(parent1, parent2)
            child = gaussian_mutation(child, varbound, GA_PARAMS['mutation_prob'])
            new_population.append(child)

        population = np.array(new_population)

    # 输出最终结果
    print("优化完成: 最终投放方案")
    print(f"最大有效遮蔽时长 (T_eff): {best_fitness_so_far:.4f} 秒")
    print("最优参数组合:")

    theta_opt, v_opt, t01, dt1, t02, dt2, t03, dt3 = best_solution_so_far
    bombs_final = [
        {'id': 1, 't_drop': t01, 't_d': dt1},
        {'id': 2, 't_drop': t02, 't_d': dt2},
        {'id': 3, 't_drop': t03, 't_d': dt3}
    ]
    bombs_final.sort(key=lambda p: p['t_drop'])

    print(f"无人机飞行方向: {np.rad2deg(theta_opt):.2f} 度")
    print(f"无人机飞行速度: {v_opt:.2f} m/s")

    for bomb in bombs_final:
        print(f"第{bomb['id']}枚烟幕弹")
        print(f"投弹时间 (t_drop): {bomb['t_drop']:.2f} 秒")
        print(f"引爆延迟 (t_d): {bomb['t_d']:.2f} 秒")
        print(f"引爆时刻:       {bomb['t_drop'] + bomb['t_d']:.2f} 秒")


if __name__ == "__main__":

    # 应用遗传算法
    print('\n遗传算法:')
    optimize_with_genetic()
```

附录 F　问题四：遗传算法求解源码

```
import numpy as np
import random
import warnings
```

```python
# 关闭不必要的警告
warnings.filterwarnings("ignore", category=RuntimeWarning)

# ===== 定义问题常量 =====
FY_INITIAL = [
    np.array([17800, 0, 1800]),
    np.array([12000, 1400, 1400]),
    np.array([6000, -3000, 700])
]
M1_START = np.array([20000, 0, 2000])
MISSILE_TARGET_POINT = np.array([0, 0, 0])
TARGET_CYLINDER_BASE_CENTER = np.array([0, 200, 0])
TARGET_CYLINDER_RADIUS = 7.0
TARGET_CYLINDER_HEIGHT = 10.0
MISSILE_SPEED = 300.0
CLOUD_FALL_SPEED = 3.0
CLOUD_RADIUS = 10.0
CLOUD_EFFECTIVE_DURATION = 20.0
G = 9.8
DRONE_SPEED_BOUNDS = (70.0, 140.0)

# 生成目标圆柱体点集
num_target_points_per_face = 100
angles = np.linspace(0, 2 * np.pi, num_target_points_per_face, endpoint=False)
x_rim = TARGET_CYLINDER_BASE_CENTER[0] + TARGET_CYLINDER_RADIUS * np.cos(angles)
y_rim = TARGET_CYLINDER_BASE_CENTER[1] + TARGET_CYLINDER_RADIUS * np.sin(angles)
points_bottom = np.vstack([x_rim, y_rim, np.full_like(x_rim, 0)]).T
points_top = np.vstack([x_rim, y_rim, np.full_like(x_rim, TARGET_CYLINDER_HEIGHT)]).T
TARGET_POINTS = np.vstack([points_bottom, points_top])

def calculate_m1_arrival_time():
    """计算导弹到达目标点的时间"""
    distance = np.linalg.norm(MISSILE_TARGET_POINT - M1_START)
    return distance / MISSILE_SPEED

M1_ARRIVAL_TIME = calculate_m1_arrival_time()

def distance_point_to_line_segment(p, a, b):
    """计算点p到线段ab的最短距离"""
    if np.all(a == b):
        return np.linalg.norm(p - a)
    ap, ab = p - a, b - a
    norm_ab_sq = np.dot(ab, ab)
    if norm_ab_sq == 0:
        return np.linalg.norm(p - a)
    t = np.dot(ap, ab) / norm_ab_sq
    if t < 0.0:
```

```
        return np.linalg.norm(p - a)
    if t > 1.0:
        return np.linalg.norm(p - b)
    return np.linalg.norm(p - (a + t * ab))

def get_cover_time(params):
    """
    计算目标被遮蔽的总时间
    :param params: 12个参数 [v1, angle1, t_fly1, t_burst1, v2, angle2, t_fly2, t_burst2, v3,
        angle3, t_fly3, t_burst3]
    :return: 目标被遮蔽总时间
    """
    # 解包参数
    v1, angle1, t_fly1, t_burst1, v2, angle2, t_fly2, t_burst2, v3, angle3, t_fly3, t_burst3 =
        params
    drone_speeds = [v1, v2, v3]
    drone_angles = [angle1, angle2, angle3]
    drone_flying_times = [t_fly1, t_fly2, t_fly3]
    bomb_burst_times = [t_burst1, t_burst2, t_burst3]

    # 计算烟雾弹爆炸时间
    bomb_detonation_times = [
        drone_flying_times[i] + bomb_burst_times[i]
        for i in range(3)
    ]

    # 检查是否所有烟雾弹都在导弹到达后爆炸（无效解）
    if all(t >= M1_ARRIVAL_TIME for t in bomb_detonation_times):
        return 0

    # 计算导弹方向向量
    missile_direction = (MISSILE_TARGET_POINT - M1_START) / np.linalg.norm(MISSILE_TARGET_POINT
        - M1_START)

    # 计算烟雾弹投放和爆炸位置
    bomb_data = []
    for i in range(3):
        v_x = drone_speeds[i] * np.cos(drone_angles[i])
        v_y = drone_speeds[i] * np.sin(drone_angles[i])
        v_drone = np.array([v_x, v_y, 0])

        # 投放位置
        drop_pos = FY_INITIAL[i] + v_drone * drone_flying_times[i]

        # 爆炸位置（考虑重力影响）
        detonation_pos = drop_pos + v_drone * bomb_burst_times[i] + 0.5 * np.array([0, 0, -G]) *
            bomb_burst_times[i]**2
```

```
    bomb_data.append({
        'detonation_time': drone_flying_times[i] + bomb_burst_times[i],
        'expiry_time': drone_flying_times[i] + bomb_burst_times[i] +
            CLOUD_EFFECTIVE_DURATION,
        'detonation_pos': detonation_pos
    })

# 时间步长和总遮蔽时间
delta_t = 0.2
cover_time = 0
t_start = min(b['detonation_time'] for b in bomb_data)
t_end = M1_ARRIVAL_TIME + 20 # 导弹到达后继续观察20秒

# 模拟导弹飞行过程
for t in np.arange(t_start, t_end, delta_t):
    missile_pos = M1_START + missile_direction * MISSILE_SPEED * t

    # 检查当前活跃的烟雾弹
    active_bombs = [
        bomb for bomb in bomb_data
        if bomb['detonation_time'] <= t <= bomb['expiry_time']
    ]

    if not active_bombs:
        continue

    # 检查所有目标点是否被遮蔽
    all_points_covered = True
    for point in TARGET_POINTS:
        point_covered = False

        # 检查该点是否被任何活跃烟雾弹遮蔽
        for bomb in active_bombs:
            # 计算烟雾当前位置（考虑下落）
            cloud_center = bomb['detonation_pos'] - np.array([0, 0, CLOUD_FALL_SPEED * (t -
                bomb['detonation_time'])])

            # 计算点到导弹-目标连线的距离
            distance = distance_point_to_line_segment(cloud_center, missile_pos, point)

            if distance <= CLOUD_RADIUS:
                point_covered = True
                break

        if not point_covered:
            all_points_covered = False
```

```
            break

        # 如果所有目标点都被遮蔽，增加遮蔽时间
        if all_points_covered:
            cover_time += delta_t

    return cover_time

# 遗传算法相关函数
def initialize_population(pop_size, bounds):
    """初始化种群"""
    population = np.zeros((pop_size, len(bounds)))
    for i in range(pop_size):
        for j in range(len(bounds)):
            population[i, j] = random.uniform(bounds[j][0], bounds[j][1])
    return population

def roulette_wheel_selection(population, fitness_scores):
    """轮盘赌选择"""
    total_fitness = sum(fitness_scores)
    if total_fitness <= 0:
        return population[np.random.choice(len(population))]
    selection_probs = [f / total_fitness for f in fitness_scores]
    return population[np.random.choice(len(population), p=selection_probs)]

def uniform_crossover(parent1, parent2):
    """均匀交叉"""
    child = parent1.copy()
    for i in range(len(parent1)):
        if random.random() < 0.5:
            child[i] = parent2[i]
    return child

def gaussian_mutation(individual, bounds, mutation_prob, mutation_scale=0.1):
    """高斯变异"""
    mutated_individual = individual.copy()
    for i in range(len(individual)):
        if random.random() < mutation_prob:
            range_width = bounds[i][1] - bounds[i][0]
            mutation = np.random.normal(0, range_width * mutation_scale)
            mutated_individual[i] += mutation
            # 确保仍在边界内
            mutated_individual[i] = np.clip(mutated_individual[i], bounds[i][0], bounds[i][1])
    return mutated_individual

def local_search(individual, bounds):
    """对单个精英个体进行局部搜索"""
```

```
    best_neighbor = individual.copy()
    best_fitness = get_cover_time(best_neighbor)

    # 在小邻域内随机扰动20次
    for _ in range(20):
        neighbor = best_neighbor.copy()

        # 微调飞行时间和起爆时间
        for j in [2, 3, 6, 7, 10, 11]: # 索引: t_fly1, t_burst1, t_fly2, t_burst2, t_fly3,
            t_burst3
            range_width = bounds[j][1] - bounds[j][0]
            perturbation = np.random.normal(0, range_width * 0.02) # 更小的扰动
            neighbor[j] += perturbation

        # 确保所有参数在边界内
        for d in range(len(neighbor)):
            neighbor[d] = np.clip(neighbor[d], bounds[d][0], bounds[d][1])

        # 检查约束: 爆炸时间必须小于导弹到达时间
        detonation_times = [
            neighbor[2] + neighbor[3], # t_fly1 + t_burst1
            neighbor[6] + neighbor[7], # t_fly2 + t_burst2
            neighbor[10] + neighbor[11] # t_fly3 + t_burst3
        ]
        if any(dt > M1_ARRIVAL_TIME for dt in detonation_times):
            continue

        neighbor_fitness = get_cover_time(neighbor)
        if neighbor_fitness > best_fitness:
            best_fitness = neighbor_fitness
            best_neighbor = neighbor

    return best_neighbor, best_fitness


if __name__ == '__main__':

    # 定义12个决策变量的边界 (每架无人机4个参数)
    varbound = [
        DRONE_SPEED_BOUNDS, (0, 2*np.pi), (0, M1_ARRIVAL_TIME), (0, M1_ARRIVAL_TIME), # FY1
        DRONE_SPEED_BOUNDS, (0, 2*np.pi), (0, M1_ARRIVAL_TIME), (0, M1_ARRIVAL_TIME), # FY2
        DRONE_SPEED_BOUNDS, (0, 2*np.pi), (0, M1_ARRIVAL_TIME), (0, M1_ARRIVAL_TIME) # FY3
    ]

    # 配置遗传算法参数
    GA_PARAMS = {
        'num_generations': 300,
```

```
    'population_size': 600,
    'elit_ratio': 0.05,   # 精英比例
    'crossover_prob': 0.8,
    'mutation_prob': 0.4,
}

# 初始化
population = initialize_population(GA_PARAMS['population_size'], varbound)
best_solution_so_far = None
best_fitness_so_far = -1

# 迭代进化
print("\n遗传算法")
for gen in range(GA_PARAMS['num_generations']):
    # 计算适应度
    fitness_scores = []
    for ind in population:
        # 检查约束: 爆炸时间必须小于导弹到达时间
        detonation_times = [
            ind[2] + ind[3], # FY1: t_fly + t_burst
            ind[6] + ind[7], # FY2
            ind[10] + ind[11] # FY3
        ]
        if any(dt > M1_ARRIVAL_TIME for dt in detonation_times):
            fitness_scores.append(0) # 无效解
        else:
            fitness_scores.append(get_cover_time(ind))

    fitness_scores = np.array(fitness_scores)

    # 寻找当前代的精英
    num_elites = int(GA_PARAMS['population_size'] * GA_PARAMS['elit_ratio'])
    elite_indices = np.argsort(fitness_scores)[-num_elites:]
    elites = population[elite_indices]

    # 局部搜索增强
    print(f" 第 {gen+1}/{GA_PARAMS['num_generations']} 代, 最佳时长:
        {np.max(fitness_scores):.4f}s. 微调 {num_elites} 个精英...")
    refined_elites = []
    for elite in elites:
        refined_sol, refined_fit = local_search(elite, varbound)
        refined_elites.append(refined_sol)

    # 如果找到全局更优解则更新
    current_gen_best_idx = np.argmax(fitness_scores)
    if fitness_scores[current_gen_best_idx] > best_fitness_so_far:
        best_fitness_so_far = fitness_scores[current_gen_best_idx]
```

```
        best_solution_so_far = population[current_gen_best_idx].copy()
        print('当前变量值:')
        print(best_solution_so_far)

    # 生成新一代
    new_population = list(refined_elites) # 精英直接进入下一代

    while len(new_population) < GA_PARAMS['population_size']:
        parent1 = roulette_wheel_selection(population, fitness_scores)
        parent2 = roulette_wheel_selection(population, fitness_scores)

        child = uniform_crossover(parent1, parent2)
        child = gaussian_mutation(child, varbound, GA_PARAMS['mutation_prob'])
        new_population.append(child)

    population = np.array(new_population)

# 输出最终结果
print(f"最大有效遮蔽时长 (T_eff): {best_fitness_so_far:.3f} 秒")
print(f"导弹到达时间: {M1_ARRIVAL_TIME:.2f} 秒")

# 提取最佳参数
params = best_solution_so_far
drones = [
    {'id': 'FY1', 'speed': params[0], 'angle': params[1], 't_fly': params[2], 't_burst':
        params[3]},
    {'id': 'FY2', 'speed': params[4], 'angle': params[5], 't_fly': params[6], 't_burst':
        params[7]},
    {'id': 'FY3', 'speed': params[8], 'angle': params[9], 't_fly': params[10], 't_burst':
        params[11]}
]

# 输出各无人机策略
for drone in drones:
    angle_deg = np.rad2deg(drone['angle'])
    detonation_time = drone['t_fly'] + drone['t_burst']
    print(f"\n{drone['id']} 投放方案")
    print(f"飞行速度: {drone['speed']:.2f} m/s")
    print(f"飞行角度: {angle_deg:.2f} 度")
    print(f"飞行时间: {drone['t_fly']:.2f} 秒")
    print(f"起爆时间: {drone['t_burst']:.2f} 秒")
    print(f"爆炸时间: {detonation_time:.2f} 秒")
    print(f"爆炸位置: {FY_INITIAL[['FY1','FY2','FY3'].index(drone['id'])] +
        drone['speed']*np.array([np.cos(drone['angle']), np.sin(drone['angle']),
        0])*drone['t_fly']}")

# 验证最终解
```

```
    final_cover_time = get_cover_time(best_solution_so_far)
    print(f"验证遮蔽时长: {final_cover_time:.3f} 秒")
```

附录 G　问题五：遗传算法求解源码

```
import numpy as np
import random
import warnings

# 关闭不必要的警告
warnings.filterwarnings("ignore", category=RuntimeWarning)

# ===== 定义问题常量 =====
FY_INITIAL = [
    np.array([17800, 0, 1800]),
    np.array([12000, 1400, 1400]),
    np.array([6000, -3000, 700]),
    np.array([11000, 2000, 1800]),
    np.array([13000, -2000, 1300])
]
M_INITIAL = [
    np.array([20000, 0, 2000]),
    np.array([19000, 600, 2100]),
    np.array([18000, -600, 1900])
]
MISSILE_TARGET_POINT = np.array([0, 0, 0])
TARGET_CYLINDER_BASE_CENTER = np.array([0, 200, 0])
TARGET_CYLINDER_RADIUS = 7.0
TARGET_CYLINDER_HEIGHT = 10.0
MISSILE_SPEED = 300.0
CLOUD_FALL_SPEED = 3.0
CLOUD_RADIUS = 10.0
CLOUD_EFFECTIVE_DURATION = 20.0
G = 9.8
DRONE_SPEED_BOUNDS = (70.0, 140.0)
NUM_SCALE = 5

# 生成目标圆柱体点集
num_target_points_per_face = 100
angles = np.linspace(0, 2 * np.pi, num_target_points_per_face, endpoint=False)
x_rim = TARGET_CYLINDER_BASE_CENTER[0] + TARGET_CYLINDER_RADIUS * np.cos(angles)
y_rim = TARGET_CYLINDER_BASE_CENTER[1] + TARGET_CYLINDER_RADIUS * np.sin(angles)
points_bottom = np.vstack([x_rim, y_rim, np.full_like(x_rim, 0)]).T
points_top = np.vstack([x_rim, y_rim, np.full_like(x_rim, TARGET_CYLINDER_HEIGHT)]).T
TARGET_POINTS = np.vstack([points_bottom, points_top])
```

```python
def calculate_missile_arrival_times():
    """计算各导弹到达目标点的时间"""
    return [np.linalg.norm(m - MISSILE_TARGET_POINT) / MISSILE_SPEED for m in M_INITIAL]

M_ARRIVAL_TIMES = calculate_missile_arrival_times()
MAX_M_ARRIVAL_TIME = max(M_ARRIVAL_TIMES)

def distance_point_to_line_segment(p, a, b):
    """计算点p到线段ab的最短距离"""
    if np.all(a == b):
        return np.linalg.norm(p - a)
    ap, ab = p - a, b - a
    norm_ab_sq = np.dot(ab, ab)
    if norm_ab_sq == 0:
        return np.linalg.norm(p - a)
    t = np.dot(ap, ab) / norm_ab_sq
    if t < 0.0:
        return np.linalg.norm(p - a)
    if t > 1.0:
        return np.linalg.norm(p - b)
    return np.linalg.norm(p - (a + t * ab))

def get_cover_time(params):
    """
    计算目标被遮蔽的总时间
    :param params: 40个参数 [v0, theta0, t_fly0, rest01, rest02, burst00, burst01, burst02,
                        v1, theta1, t_fly1, rest11, rest12, burst10, burst11, burst12,
                        ...]
    :return: 目标被遮蔽总时间
    """
    # 解包参数 (5架无人机, 每架8个参数)
    drone_params = [params[i*8:(i+1)*8] for i in range(5)]

    # 组织成get_cover_time需要的格式
    drone_speeds, drone_angles, drone_flying_times = [], [], []
    rest_times, bomb_burst_times = [], []

    for param in drone_params:
        v, theta, t_fly, rest1, rest2, burst1, burst2, burst3 = param
        drone_speeds.append(v)
        drone_angles.append(theta)
        drone_flying_times.append(t_fly)
        rest_times.append([rest1, rest2])
        bomb_burst_times.append([burst1, burst2, burst3])

    return _get_cover_time([
```

```
        drone_speeds,
        drone_angles,
        drone_flying_times,
        rest_times,
        bomb_burst_times
    ])

def _get_cover_time(particle, output=False):
    """内部函数，计算遮蔽时间"""
    drone_speeds, drone_angles, drone_flying_time, rest_time, bomb_burst_time = particle

    # 无人机投弹时刻：5×3
    drone_drop_times = [
        [
            drone_flying_time[i],
            drone_flying_time[i] + rest_time[i][0],
            drone_flying_time[i] + rest_time[i][0] + rest_time[i][1]
        ] for i in range(5)
    ]

    # 烟雾弹爆炸时刻：5×3
    bomb_detonation_times = [
        [
            drone_flying_time[i] + bomb_burst_time[i][0],
            drone_flying_time[i] + rest_time[i][0] + bomb_burst_time[i][1],
            drone_flying_time[i] + rest_time[i][0] + rest_time[i][1] + bomb_burst_time[i][2]
        ] for i in range(5)
    ]

    # 检查是否所有爆炸均在导弹命中后
    if all(
        det_time >= MAX_M_ARRIVAL_TIME
        for row in bomb_detonation_times for det_time in row
    ):
        return 0

    bomb_data = []
    for j in range(5):
        v_x = drone_speeds[j] * np.cos(drone_angles[j])
        v_y = drone_speeds[j] * np.sin(drone_angles[j])
        v_drone = np.array([v_x, v_y, 0])

        for i in range(3):
            t_drop = drone_drop_times[j][i]
            t_det = bomb_detonation_times[j][i]
            t_delay = bomb_burst_time[j][i]
```

```
        p_drop = FY_INITIAL[j] + v_drone * t_drop
        p_detonation = p_drop + v_drone * t_delay + 0.5 * np.array([0, 0, -G]) * t_delay**2

        bomb_data.append({
            'drone_id': j,
            'bomb_id': i,
            't_det': t_det,
            't_exp': t_det + CLOUD_EFFECTIVE_DURATION,
            'p_det': p_detonation,
            'p_drop': p_drop
        })

# 导弹方向向量
m_dir = [(MISSILE_TARGET_POINT - m) / np.linalg.norm(MISSILE_TARGET_POINT - m) for m in
    M_INITIAL]

# 时间设置
t_start = min(b['t_det'] for b in bomb_data) if bomb_data else 0
t_end = MAX_M_ARRIVAL_TIME + CLOUD_EFFECTIVE_DURATION
dt = 0.2

cover_time = 0.0

# 模拟时间
for t in np.arange(t_start, t_end, dt):
    p_m_t = [M_INITIAL[k] + m_dir[k] * MISSILE_SPEED * t for k in range(3)]

    # 当前活跃的烟雾弹
    active_bombs = [b for b in bomb_data if b['t_det'] <= t <= b['t_exp']]
    if not active_bombs:
        continue

    # 检查导弹是否被遮蔽
    for k in range(3): # 每枚导弹
        all_target_points_concealed = True
        for p_target in TARGET_POINTS: # 每个目标点
            point_is_concealed = False
            for active_bomb in active_bombs: # 每个活跃烟雾弹
                p_cloud_center = active_bomb['p_det'] - np.array([0, 0, CLOUD_FALL_SPEED * (t
                    - active_bomb['t_det'])])
                if distance_point_to_line_segment(p_cloud_center, p_m_t[k], p_target) <=
                    CLOUD_RADIUS:
                    point_is_concealed = True
                    break

            if not point_is_concealed:
                all_target_points_concealed = False
```

```
                break

        if all_target_points_concealed:
            cover_time += dt

    return cover_time

# 遗传算法相关函数
def initialize_population(pop_size, bounds):
    """初始化种群"""
    population = np.zeros((pop_size, len(bounds)))
    for i in range(pop_size):
        for j in range(len(bounds)):
            # 40个参数 [v0, theta0, t_fly0, rest01, rest02, burst00, burst01, burst02,
            #          v1, theta1, t_fly1, rest11, rest12, burst10, burst11, burst12,
            # 对于投弹间隔参数（索引3,4,11,12,19,20,27,28,35,36），确保 1，且应用缩放系数
            if j in [3,4,11,12,19,20,27,28,35,36]:
                population[i, j] = random.uniform(max(1.0, bounds[j][0]), bounds[j][1]/NUM_SCALE)
            # 对于起爆时间参数，应用缩放系数
            elif j in [2,5,6,7,10,13,14,15,18,21,22,23,26,29,30,31,34,37,38,39]:
                population[i, j] = random.uniform(bounds[j][0], bounds[j][1]/NUM_SCALE)
            else:
                population[i, j] = random.uniform(bounds[j][0], bounds[j][1])
    return population

def roulette_wheel_selection(population, fitness_scores):
    """轮盘赌选择"""
    total_fitness = sum(fitness_scores)
    if total_fitness <= 0:
        return population[np.random.choice(len(population))]
    selection_probs = [f / total_fitness for f in fitness_scores]
    return population[np.random.choice(len(population), p=selection_probs)]

def uniform_crossover(parent1, parent2):
    """均匀交叉"""
    child = parent1.copy()
    for i in range(len(parent1)):
        if random.random() < 0.5:
            child[i] = parent2[i]
    return child

def gaussian_mutation(individual, bounds, mutation_prob, mutation_scale=0.1):
    """高斯变异"""
    mutated_individual = individual.copy()
    for i in range(len(individual)):
        if random.random() < mutation_prob:
            range_width = bounds[i][1] - bounds[i][0]
```

```
            mutation = np.random.normal(0, range_width * mutation_scale)
            mutated_individual[i] += mutation

            # 确保参数在边界内
            mutated_individual[i] = np.clip(mutated_individual[i], bounds[i][0], bounds[i][1])

            # 对于投弹间隔参数，确保 1秒
            if i in [3,4,11,12,19,20,27,28,35,36]:
                mutated_individual[i] = max(1.0, mutated_individual[i])

    return mutated_individual

def local_search(individual, bounds):
    """对单个精英个体进行局部搜索"""
    best_neighbor = individual.copy()
    best_fitness = get_cover_time(best_neighbor)

    # 在小邻域内随机扰动30次
    for _ in range(30):
        neighbor = best_neighbor.copy()

        # 微调飞行时间、投弹间隔和起爆时间
        for j in range(len(individual)):
            # 只调整时间相关参数（索引2,3,4,5,6,7,10,11,12,13,14,15,...）
            if j in
                [2,3,4,5,6,7,10,11,12,13,14,15,18,19,20,21,22,23,26,27,28,29,30,31,34,35,36,37,38,39]:
                range_width = bounds[j][1] - bounds[j][0]
                perturbation = np.random.normal(0, range_width * 0.02)
                neighbor[j] += perturbation

                # 确保参数在边界内
                neighbor[j] = np.clip(neighbor[j], bounds[j][0], bounds[j][1])

                # 对于投弹间隔参数，确保 1秒
                if j in [3,4,11,12,19,20,27,28,35,36]:
                    neighbor[j] = max(1.0, neighbor[j])

        # 检查爆炸时间约束
        valid = True
        for i in range(5):
            t_fly = neighbor[i*8+2]
            rest1 = neighbor[i*8+3]
            rest2 = neighbor[i*8+4]
            burst1 = neighbor[i*8+5]
            burst2 = neighbor[i*8+6]
            burst3 = neighbor[i*8+7]
```

```
            detonation_times = [
                t_fly + burst1,
                t_fly + rest1 + burst2,
                t_fly + rest1 + rest2 + burst3
            ]

            if any(dt > MAX_M_ARRIVAL_TIME for dt in detonation_times):
                valid = False
                break

            # 检查投弹间隔是否 1秒
            if rest1 < 1.0 or rest2 < 1.0:
                valid = False
                break

        if not valid:
            continue

        neighbor_fitness = get_cover_time(neighbor)
        if neighbor_fitness > best_fitness:
            best_fitness = neighbor_fitness
            best_neighbor = neighbor

    return best_neighbor, best_fitness


if __name__ == '__main__':
    print(f"导弹到达时间: M1={M_ARRIVAL_TIMES[0]:.2f}s, M2={M_ARRIVAL_TIMES[1]:.2f}s,
        M3={M_ARRIVAL_TIMES[2]:.2f}s")
    print(f"最晚导弹到达时间: {MAX_M_ARRIVAL_TIME:.2f}s")

    # 定义40个决策变量的边界 (5架无人机, 每架8个参数)
    T_max = MAX_M_ARRIVAL_TIME
    varbound = []
    for _ in range(5): # 每架无人机
        varbound.append(DRONE_SPEED_BOUNDS) # 速度
        varbound.append((0, 2*np.pi))     # 角度
        varbound.append((0, T_max))       # 飞行时间
        varbound.append((1.0, T_max))     # 投弹间隔1 (必须 1秒)
        varbound.append((1.0, T_max))     # 投弹间隔2 (必须 1秒)
        varbound.append((0, T_max))       # 第一枚弹起爆时间
        varbound.append((0, T_max))       # 第二枚弹起爆时间
        varbound.append((0, T_max))       # 第三枚弹起爆时间

    # 配置遗传算法参数
    GA_PARAMS = {
        'num_generations': 200, # 增加代数
```

```
    'population_size': 1000, # 增大种群
    'elit_ratio': 0.01,  # 精英比例
    'crossover_prob': 0.7,
    'mutation_prob': 0.4,
}

# 初始化
population = initialize_population(GA_PARAMS['population_size'], varbound)
best_solution_so_far = None
best_fitness_so_far = -1

# 迭代进化
print("\n遗传算法")
for gen in range(GA_PARAMS['num_generations']):
    # 计算适应度
    fitness_scores = []
    for ind in population:
        # 检查约束：爆炸时间必须小于导弹到达时间，投弹间隔>=1秒
        valid = True
        for i in range(5):
            t_fly = ind[i*8+2]
            rest1 = ind[i*8+3]
            rest2 = ind[i*8+4]
            burst1 = ind[i*8+5]
            burst2 = ind[i*8+6]
            burst3 = ind[i*8+7]

            detonation_times = [
                t_fly + burst1,
                t_fly + rest1 + burst2,
                t_fly + rest1 + rest2 + burst3
            ]

            if any(dt > MAX_M_ARRIVAL_TIME for dt in detonation_times):
                valid = False
                break

            # 检查投弹间隔是否>=1秒
            if rest1 < 1.0 or rest2 < 1.0:
                valid = False
                break

        if not valid:
            fitness_scores.append(0) # 无效解
        else:
            fitness_scores.append(get_cover_time(ind))
```

```
    fitness_scores = np.array(fitness_scores)

    # 寻找当前代的精英
    num_elites = int(GA_PARAMS['population_size'] * GA_PARAMS['elit_ratio'])
    elite_indices = np.argsort(fitness_scores)[-num_elites:]
    elites = population[elite_indices]

    # 局部搜索增强
    current_best = np.max(fitness_scores)
    print(f" 第 {gen+1}/{GA_PARAMS['num_generations']} 代，最佳时长: {current_best:.4f}s.
        微调 {num_elites} 个精英...")
    refined_elites = []
    for elite in elites:
        refined_sol, refined_fit = local_search(elite, varbound)
        refined_elites.append(refined_sol)

    # 如果找到全局更优解则更新
    current_gen_best_idx = np.argmax(fitness_scores)
    if fitness_scores[current_gen_best_idx] > best_fitness_so_far:
        best_fitness_so_far = fitness_scores[current_gen_best_idx]
        best_solution_so_far = population[current_gen_best_idx].copy()
        print(best_solution_so_far)

    # 生成新一代
    new_population = list(refined_elites) # 精英直接进入下一代

    while len(new_population) < GA_PARAMS['population_size']:
        parent1 = roulette_wheel_selection(population, fitness_scores)
        parent2 = roulette_wheel_selection(population, fitness_scores)

        child = uniform_crossover(parent1, parent2)
        child = gaussian_mutation(child, varbound, GA_PARAMS['mutation_prob'])
        new_population.append(child)

    population = np.array(new_population)

# 输出最终结果
print(f"最大有效遮蔽时长 (T_eff): {best_fitness_so_far:.3f} 秒")
print(f"导弹到达时间: M1={M_ARRIVAL_TIMES[0]:.2f}s, M2={M_ARRIVAL_TIMES[1]:.2f}s,
    M3={M_ARRIVAL_TIMES[2]:.2f}s")

# 提取最佳参数
params = best_solution_so_far
drones = []

# 组织无人机参数
for i in range(5):
```

```
    start_idx = i * 8
    drone_params = {
        'id': f'FY{i+1}',
        'speed': params[start_idx],
        'angle': params[start_idx+1],
        't_fly': params[start_idx+2],
        'rest_time': [params[start_idx+3], params[start_idx+4]],
        'burst_time': [params[start_idx+5], params[start_idx+6], params[start_idx+7]]
    }
    drones.append(drone_params)

# 输出各无人机策略
for drone in drones:
    angle_deg = np.rad2deg(drone['angle'])
    print(f"\n{drone['id']} 投放方案")
    print(f"飞行速度: {drone['speed']:.2f} m/s")
    print(f"飞行角度: {angle_deg:.2f} 度")
    print(f"初始飞行时间: {drone['t_fly']:.2f} 秒")
    print(f"投弹间隔: {drone['rest_time'][0]:.2f}秒, {drone['rest_time'][1]:.2f}秒")
    print(f"三枚弹起爆时间: {drone['burst_time'][0]:.2f}秒, {drone['burst_time'][1]:.2f}秒,
        {drone['burst_time'][2]:.2f}秒")

    # 计算投弹和爆炸时间
    drop_times = [
        drone['t_fly'],
        drone['t_fly'] + drone['rest_time'][0],
        drone['t_fly'] + drone['rest_time'][0] + drone['rest_time'][1]
    ]

    detonation_times = [
        drop_times[0] + drone['burst_time'][0],
        drop_times[1] + drone['burst_time'][1],
        drop_times[2] + drone['burst_time'][2]
    ]

    print(f"投弹时间: {drop_times[0]:.2f}秒, {drop_times[1]:.2f}秒, {drop_times[2]:.2f}秒")
    print(f"爆炸时间: {detonation_times[0]:.2f}秒, {detonation_times[1]:.2f}秒,
        {detonation_times[2]:.2f}秒")

# 验证最终解
final_cover_time = get_cover_time(best_solution_so_far)
print(f"验证遮蔽时长: {final_cover_time:.3f} 秒")
```